From simple ML to Responsible AI-A journey

HOANG KHANG PHAN

Foreword

This book serves as a chronicle of a transformative journey—an intellectual odyssey that begins in the structured reality of Biomedical Science and extends into the abstract, rapidly evolving frontiers of Artificial Intelligence. It documents the transition of a domain expert learning to harness the power of algorithms to solve the complex mysteries of life sciences.

In the chapters that follow, we traverse the full spectrum of modern computational intelligence. We begin with the mathematical foundations of Machine Learning and Deep Learning, stripping away the complexity to reveal the core mechanics that drive these systems. From there, we advance to the cutting edge, exploring the nuances of Large Language Models, Vision Transformers, and the bridging of modalities through architectures like CLIP and JEPA.

Crucially, this book argues that raw performance is no longer sufficient. We delve into Interpretability (SHAP, LIME) and Uncertainty Quantification, maintaining that in high-stakes fields like healthcare, the ability to trust and understand a model is just as vital as its accuracy.

Furthermore, I offer my own perspective on the future—a set of predictions on how AI will reshape the biomedical landscape. This book is more than a technical guide; it is a reflection on a long and challenging path of discovery, and a summary of hard-won experiences. It is my sincere hope that this road map serves as a beacon for other researchers standing at this same intersection, helping them navigate the exciting convergence of biology and AI.

In addition, this handbook contains worth researching topics which I denote as in the "**" brackets. These topic, in my opinion, a underdone topic, or a foundation model that can be utilize into many other field. So if you interested in some of that topics, research about it!

Finally, I want to thanks ... for comment and proof-read this book.

Contents

Chapter 1

Machine Learning fundamentals

1.1 Linear Regression

1.1.1 Conceptual Framework

Linear Regression[3] is the "hello world" of machine learning, yet it remains a powerful tool for predictive analysis. Fundamentally, it assumes a linear relationship between the input variables (features) and the single output variable (target).

If we have a single feature x, we are fitting a line. If we have multiple features, we are fitting a hyperplane in n-dimensional space.

1.1.2 Mathematical Formulation

Let our training set consist of m samples. For a given sample i, let $x^{(i)}$ denote the input feature vector and $y^{(i)}$ denote the actual target value.

The hypothesis function $h_\theta(x)$ is defined as:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \tag{1.1}$$

Here:

- θ_0 is the **bias** (or intercept) term.
- $\theta_1, \ldots, \theta_n$ are the **weights** (or coefficients) associated with each feature.

To simplify notation, we introduce $x_0 = 1$ for all samples, allowing us to write the equation in vectorized form:

$$h_\theta(x) = \theta^T x \tag{1.2}$$

where θ and x are $(n+1) \times 1$ vectors.

1.1.3 The Cost Function: Mean Squared Error (MSE)

We need a metric to evaluate how well our line fits the data. We define the **residual** as the difference between the predicted value and the actual value:

$$\text{Error}^{(i)} = h_\theta(x^{(i)}) - y^{(i)}$$

To penalize large errors, we square this difference. Averaging this over all training examples gives us the **Mean Squared Error (MSE)** cost function, denoted as $J(\theta)$:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2 \tag{1.3}$$

Note: The factor of $\frac{1}{2}$ is a mathematical convenience that cancels out when we take the derivative during gradient descent.

1.1.4 Optimization: Gradient Descent

The cost function $J(\theta)$ for linear regression is **convex**, meaning it has a single global minimum (a bowl shape). We use an iterative algorithm called Gradient Descent to find the optimal θ.

The update rule for any parameter θ_j is:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \tag{1.4}$$

where α is the **learning rate**.

Deriving the partial derivative:

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

$$= \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

1.1.5 Use Cases and Limitations

Common Use Cases:

- **Economics:** Predicting GDP growth based on interest rates and unemployment.
- **Real Estate:** Estimating property value based on square footage and location.
- **Manufacturing:** Forecasting machine lifespan based on usage hours and load.

Key Assumptions:

1. **Linearity:** The relationship between X and Y is linear.
2. **Homoscedasticity:** The variance of residual terms is constant.
3. **No Multicollinearity:** Independent variables are not highly correlated with each other.

1.2 Logistic Regression

1.2.1 Conceptual Framework

Despite its name, Logistic Regression [4] is a **classification** algorithm. It is used when the response variable y is categorical, most commonly binary (0 or 1).

If we attempted to use Linear Regression for classification, the predicted y could range from $-\infty$ to $+\infty$. This is problematic because probability must be bounded between $[0, 1]$. To solve this, we wrap our linear equation in a "squashing" function called the **Sigmoid Function**.

1.2.2 The Sigmoid Activation Function

The Sigmoid function, denoted as $\sigma(z)$, maps any real number to the $(0, 1)$ interval:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{1.5}$$

Substituting our linear equation $z = \theta^T x$, our new hypothesis becomes:

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}} \tag{1.6}$$

Probabilistic Interpretation

The output $h_\theta(x)$ is interpreted as the probability that the input x belongs to the positive class ($y = 1$):

$$h_\theta(x) = P(y = 1|x; \theta)$$

Consequently, the probability of the negative class is:

$$P(y = 0|x; \theta) = 1 - h_\theta(x)$$

1.2.3 The Decision Boundary

To make a discrete classification, we apply a threshold (typically 0.5):

- Predict $y = 1$ if $h_\theta(x) \geq 0.5$ (which implies $\theta^T x \geq 0$)
- Predict $y = 0$ if $h_\theta(x) < 0.5$ (which implies $\theta^T x < 0$)

The line (or hyperplane) defined by $\theta^T x = 0$ is known as the **Decision Boundary**.

1.2.4 Cost Function: Binary Cross-Entropy (Log Loss)

If we used Mean Squared Error for Logistic Regression, the non-linear Sigmoid function would result in a "wavy" cost function with many local minima (non-convex). Gradient Descent would fail to find the global optimum.

Instead, we use **Log Loss**. We want the algorithm to be penalized heavily if it predicts a probability close to 0 when the actual label is 1, and vice versa.

$$Cost(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases} \tag{1.7}$$

This can be combined into a single equation for all m examples:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right] \tag{1.8}$$

1.2.5 Use Cases

Logistic Regression is the industry standard for binary classification tasks where probability estimation is required:

- **Credit Scoring:** Predicting the probability of default (Default/No Default).
- **Marketing:** Predicting if a user will click an ad (Click/No Click).
- **Medicine:** Predicting the presence of a disease based on symptoms (Positive/Negative).

1.3 Decision Trees

1.3.1 Problem Statement

In previous chapters, we assumed data could be separated by a smooth, continuous line or curve. However, real-world data is often "jagged" and non-linear. The goal of a Decision

Tree [5] is to partition the feature space $\mathcal{X}$ into distinct, non-overlapping regions R_m.

1.3.2 The Mathematical Definition of Impurity

To automate the splitting process, we first need a mathematical function that quantifies the "disorder" or "impurity" of a dataset. Let S be a set of training examples, and let p_k be the probability (or proportion) of class k in that set.

We define a generic **impurity function** $I(S)$ which must satisfy three properties:

1. $I(S) = 0$ if the set is pure (all examples belong to one class).
2. $I(S)$ is maximized when the class distribution is uniform (maximum uncertainty).
3. $I(S)$ is symmetric with respect to the classes.

There are two standard functions used for $I(S)$:

Shannon's Entropy (H)

Derived from Information Theory, Entropy measures the expected amount of "surprise" in the data.

$$I_{Entropy}(S) = -\sum_{k=1}^{K} p_k \log_2(p_k) \tag{1.9}$$

Interpretation: If a node is pure ($p_1 = 1, p_2 = 0$), $\log_2(1) = 0$, so Entropy is 0. If a node is evenly split ($p_1 = 0.5, p_2 = 0.5$), Entropy is 1.

Gini Impurity (G)

Gini measures the probability that a randomly chosen element from the set would be incorrectly labeled if it were labeled according to the distribution of labels in the set.

$$I_{Gini}(S) = 1 - \sum_{k=1}^{K} p_k^2 \tag{1.10}$$

Interpretation: Gini is computationally faster than Entropy because it avoids logarithmic calculations. It ranges from 0 (pure) to 0.5 (maximum impurity for binary classification).

1.3.3 The Objective: Maximizing Information Gain

The learning algorithm attempts to construct a tree by recursively splitting the data. At each step, it selects the split that maximizes the **Information Gain (IG)**.

Information Gain is defined as the reduction in impurity after splitting parent set S_p into child sets S_{left} and S_{right}:

$$IG = I(S_p) - \underbrace{\left(\frac{N_{left}}{N_p} I(S_{left}) + \frac{N_{right}}{N_p} I(S_{right}) \right)}_{\text{Weighted Average Impurity of Children}} \tag{1.11}$$

Where:

- $I(\cdot)$ is the chosen impurity function (Entropy or Gini).
- N_p is the total number of samples in the parent node.
- N_{left}, N_{right} are the number of samples in the child nodes.

1.4 Random [1]

1.4.1 Problem Statement

How do we maintain the non-linear flexibility of Decision Trees while solving the High Variance/Overfitting problem?

We cannot simply stop the tree from growing (pruning), as this increases Bias (underfitting). Instead, we turn to **Ensemble Learning**. The intuition is that while one tree might be "noisy" and "wrong," the average of many distinct trees will be "stable" and "correct."

1.4.2 Mathematical Solution: Variance Reduction

Let us treat the prediction of a single tree as a random variable T. If we create an ensemble of B trees and average their predictions:

$$\bar{T} = \frac{1}{B} \sum_{i=1}^{B} T_i$$

The variance of this average is given by the general formula for the variance of the sum of correlated variables:

$$\text{Var}(\bar{T}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{1.12}$$

Where:

- σ^2 is the variance of a single tree.
- ρ is the correlation between any two trees in the forest.

- B is the number of trees.

Analysis of the Equation:

1. As $B \to \infty$, the second term $\frac{1-\rho}{B}\sigma^2$ disappears.
2. The total variance converges to $\rho\sigma^2$.

The RF Strategy: To minimize error, we must reduce ρ (correlation between trees) without increasing σ^2 (bias of individual trees). Random Forests achieve this via two specific mechanisms.

1.4.3 Mechanism 1: Bagging (Bootstrap Aggregating)

If we train every tree on the exact same dataset, they will all learn the exact same structure ($\rho = 1$), and the ensemble will effectively be just one tree.

To de-correlate them, we use **Bootstrapping**:

- We create B different datasets by sampling from the original data D uniformly *with replacement*.
- Each tree sees a slightly different view of the world.

This reduces ρ slightly, but not enough if strong features exist.

1.4.4 Mechanism 2: Feature Randomness

If one feature is very strong (e.g., "Income" for loan prediction), every bootstrapped tree will still choose to split on "Income" at the root. The trees will remain structurally similar (high ρ).

Random Forests introduce a constraint: **At each split, the algorithm is restricted to searching only a random subset of m features.** (Typically $m = \sqrt{\text{Total Features}}$).

- This forces trees to consider other, less dominant features.
- It significantly decorrelates the trees (lowers ρ).
- Even though individual trees become slightly "worse" (higher bias), the ensemble variance drops dramatically, leading to a lower total error.

1.5 Support Vector Machines

1.5.1 Introduction and Problem Statement

In the previous sections on Logistic Regression and Perceptrons, we focused on finding a hyperplane that separates two classes. If the data is linearly separable, however, there are infinite such hyperplanes.

Consider the dataset $D = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$ where $y \in \{-1, +1\}$. Logistic Regression chooses a hyperplane based on probabilistic confidence, often yielding a boundary close to the data points. This is risky: a small perturbation in a single data point could cause a misclassification.

The Core Problem: Among the infinite set of separating hyperplanes, which one is the most robust? We define robustness by the **Margin**: the distance between the hyperplane and the nearest data point of either class. The goal of the Support Vector Machine (SVM) [6] is to find the unique hyperplane that **maximizes this margin**.

1.5.2 The Primal Formulation: Geometry of the Margin

A hyperplane is defined by the equation:

$$w^T x + b = 0 \tag{1.13}$$

where $w \in \mathbb{R}^n$ is the normal vector to the hyperplane.

The perpendicular distance d from any point $x^{(i)}$ to the hyperplane is given by:

$$d_i = \frac{|w^T x^{(i)} + b|}{||w||} \tag{1.14}$$

To fix the scale of w and b, we impose a canonical constraint. We require that for the data point closest to the hyperplane (the "Support Vector"), the value of $|w^T x + b|$ is a. In this case, for simplification, we choose $a = 1$. Thus we have:

$$y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad \forall i \tag{1.15}$$

Under this constraint, the distance to the closest point is $\frac{1}{||w||}$. The total margin (gap between classes) is $\frac{2}{||w||}$. To maximize the margin, we must **minimize** $||w||$. Mathematically, it is equivalent (and easier) to minimize $\frac{1}{2}||w||^2$.

This yields the **Primal Optimization Problem**:

$$\begin{aligned} \underset{w,b}{\text{minimize}} \quad & \frac{1}{2}||w||^2 \\ \text{subject to} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad \forall i = 1, \ldots, m \end{aligned} \tag{1.16}$$

This is a convex quadratic programming problem with linear constraints.

1.5.3 The Lagrangian Dual Formulation

Solving the primal problem directly can be computationally expensive and does not easily allow for non-linear boundaries. We transition to the **Dual Form** using Lagrange Multipliers.

We construct the Lagrangian $\mathcal{L}(w, b, \alpha)$ by introducing multipliers $\alpha_i \geq 0$:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}||w||^2 - \sum_{i=1}^{m} \alpha_i \left[y^{(i)}(w^T x^{(i)} + b) - 1 \right] \tag{1.17}$$

To find the minimum with respect to w and b, we set the partial derivatives to zero:

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)} = 0 \implies w = \sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)} \tag{1.18}$$

$$\nabla_b \mathcal{L} = -\sum_{i=1}^{m} \alpha_i y^{(i)} = 0 \implies \sum_{i=1}^{m} \alpha_i y^{(i)} = 0 \tag{1.19}$$

Substituting these back into the Lagrangian eliminates w and b, yielding the **Dual Objective Function**:

$$\begin{aligned} \underset{\alpha}{\text{maximize}} \quad & \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle \\ \text{subject to} \quad & \alpha_i \geq 0, \quad \sum_{i=1}^{m} \alpha_i y^{(i)} = 0 \end{aligned} \tag{1.20}$$

Crucial Observation: The optimization depends *only* on the dot product (inner product) between pairs of samples, $\langle x^{(i)}, x^{(j)} \rangle$.

1.5.4 The Kernel Trick

The dependence on dot products allows us to handle non-linearly separable data efficiently. Instead of manually mapping data to a high-dimensional space via $\phi(x)$, we replace the dot product with a **Kernel Function** $K(x^{(i)}, x^{(j)})$.

$$K(x^{(i)}, x^{(j)}) = \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$$

This allows the SVM to find a linear boundary in an infinite-dimensional space without ever computing the coordinates in that space. Common Kernels include:

- **Polynomial Kernel:** $K(x, z) = (x^T z + c)^d$
- **Radial Basis Function (RBF):** $K(x, z) = \exp\left(-\frac{||x-z||^2}{2\sigma^2}\right)$

1.5.5 Handling Outliers: The Soft Margin

Real-world data is noisy. If we strictly enforce $y^{(i)}(w^T x^{(i)} + b) \geq 1$, a single outlier can ruin the model. We introduce **Slack Variables** $\xi_i \geq 0$ to allow some points to violate the margin.

The optimization becomes:

$$\min_{w,b,\xi} \left(\frac{1}{2}||w||^2 + C \sum_{i=1}^{m} \xi_i \right) \tag{1.21}$$

Subject to: $y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i$

The hyperparameter C controls the trade-off:

- **Large C:** Heavily penalizes error. Results in a narrow margin and potential overfitting.
- **Small C:** Tolerates more error. Results in a wider margin and potentially better generalization.

1.5.6 A Concrete 2D Example

To visualize the mathematics, let us consider a simple 2D dataset that is linearly separable.

The Dataset: Suppose we have three training points:

- **Positive Class ($y = +1$):** Two points at $x^{(1)} = [3, 3]$ and $x^{(2)} = [4, 3]$.
- **Negative Class ($y = -1$):** One point at $x^{(3)} = [1, 1]$.

Step 1: Identifying Support Vectors Intuitively, the boundary will lie between the closest points of the opposing classes. The points closest to the "gap" are $x^{(1)} = [3, 3]$ and $x^{(3)} = [1, 1]$. These are our **Support Vectors**. The point $x^{(2)}$ is "behind" $x^{(1)}$ and does not affect the margin.

Step 2: Constructing the Canonical Equations For support vectors, the constraint holds with equality: $y^{(i)}(w^T x^{(i)} + b) = 1$. Let $w = [w_1, w_2]$. We set up the system of equations:

1. For $x^{(1)}$ $(y = +1)$:

$$1 \cdot (w_1(3) + w_2(3) + b) = 1 \implies 3w_1 + 3w_2 + b = 1$$

2. For $x^{(3)}$ $(y = -1)$:

$$-1 \cdot (w_1(1) + w_2(1) + b) = 1 \implies w_1 + w_2 + b = -1$$

Step 3: Solving for Weights (w) and Bias (b) Subtracting equation (2) from equation (1):

$$(3w_1 + 3w_2 + b) - (w_1 + w_2 + b) = 1 - (-1)$$

$$2w_1 + 2w_2 = 2 \implies w_1 + w_2 = 1$$

By symmetry of the points along the line $y = x$, we can infer $w_1 = w_2$. Thus:

$$2w_1 = 1 \implies w_1 = 0.5, \quad w_2 = 0.5$$

$$w = [0.5, 0.5]$$

Substituting back to find b:

$$0.5 + 0.5 + b = -1 \implies 1 + b = -1 \implies b = -2$$

Step 4: The Decision Boundary The hyperplane equation $w^T x + b = 0$ becomes:

$$0.5x_1 + 0.5x_2 - 2 = 0$$

$$x_1 + x_2 = 4$$

This represents a line passing through $(2, 2)$ with a slope of -1, perfectly separating the classes.

Step 5: Calculating the Margin Width The margin is given by $\frac{2}{||w||}$. First, calculate the norm of w:

$$||w|| = \sqrt{0.5^2 + 0.5^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.707$$

The geometric margin is:

$$\text{Margin} = \frac{2}{\sqrt{0.5}} = 2\sqrt{2} \approx 2.828$$

This calculation confirms that by finding the specific w that satisfies the constraints for the support vectors, we have mathematically derived the widest possible street separating the two classes.

1.5.7 Geometric Interpretation: The Direction of w

You might notice an interesting geometric property in the example above. The vector connecting the negative support vector $x^{(2)}$ to the positive support vector $x^{(1)}$ is:

$$x^{(1)} - x^{(2)} = [3, 3] - [1, 1] = [2, 2]$$

Our calculated weight vector was $w = [0.5, 0.5]$. Notice that w points in the **exact same direction** as the difference between the support vectors.

Why does this happen?

This is not a coincidence; it is a fundamental property of SVMs derived from the Lagrangian Dual. Recall the equation for w from the Dual Formulation:

$$w = \sum_{i=1}^{m} \alpha_i y^{(i)} x^{(i)} \tag{1.22}$$

In our simple example, we only have two active support vectors ($x^{(1)}$ and $x^{(2)}$) with the constraint $\sum \alpha_i y^{(i)} = 0$, which implies $\alpha_1 = \alpha_2 = \alpha$.

Substituting these into the equation:

$$w = \alpha(1)x^{(1)} + \alpha(-1)x^{(2)}$$

$$w = \alpha(x^{(1)} - x^{(2)})$$

This proves that the optimal normal vector w is perfectly aligned with the vector difference between the support vectors.

- **Direction:** w points perpendicular to the decision boundary, directly from the negative class towards the positive class.
- **Magnitude:** The length $||w||$ is scaled specifically to satisfy the margin width constraint.

1.6 Bayesian Foundations

At the heart of these methods lies **Bayes' Theorem**, which allows us to invert probabilities—inferring the hidden cause from an observed effect.

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)} \tag{1.23}$$

Where H is the Hypothesis (Hidden State) and E is the Evidence (Observation). In HMMs, we extend this concept to chains of events occurring over time.

1.7 Hidden Markov Models (HMM)

1.7.1 Conceptual Framework

An HMM models [7] a system as a stochastic process consisting of two parallel sequences:

1. **Hidden States (Z):** A sequence $z_1, z_2, \ldots, z_T$ that we cannot observe directly.
2. **Observations (X):** A sequence $x_1, x_2, \ldots, x_T$ generated by the hidden states.

1.7.2 The Two Fundamental Assumptions

To make the math tractable, HMMs rely on two strong independence assumptions:

1. The First-Order Markov Assumption: The probability of the next state depends *only* on the current state, not on the entire history.

$$P(z_{t+1}|z_t, z_{t-1}, \ldots, z_1) = P(z_{t+1}|z_t) \tag{1.24}$$

2. The Output Independence Assumption: The observation at time t depends *only* on the hidden state at time t.

$$P(x_t|z_t, \ldots, x_{t-1}) = P(x_t|z_t) \tag{1.25}$$

1.7.3 Model Parameters (λ)

A complete HMM is defined by the triplet $\lambda = (A, B, \pi)$:

- **Transition Matrix (A):** $A_{ij} = P(z_{t+1} = j \mid z_t = i)$. The probability of moving from state i to state j.
- **Emission Matrix (B):** $B_j(k) = P(x_t = k \mid z_t = j)$. The probability of observing symbol k while in state j.
- **Initial Probabilities (π):** $\pi_i = P(z_1 = i)$.

1.8 The Decoding Problem and The Viterbi Algorithm[2]

1.8.1 Problem Statement

Given a model λ and a sequence of observations $X = \{x_1, x_2, \ldots, x_T\}$, we want to find the single most likely sequence of hidden states $Z^* = \{z_1^*, \ldots, z_T^*\}$.

Mathematically, we maximize the joint probability:

$$Z^* = argmax_Z P(Z, X|\lambda) \tag{1.26}$$

1.8.2 Why Brute Force Fails

If there are N possible hidden states and the sequence length is T, there are N^T possible state sequences. For a small problem with $N = 10$ states and $T = 100$ time steps, the number of paths is 10^{100} (more than the atoms in the universe). Brute force is impossible.

1.8.3 The Viterbi Solution (Dynamic Programming)

The Viterbi algorithm reduces this complexity from $O(N^T)$ to $O(N^2T)$ using Dynamic Programming. We define the variable $\delta_t(j)$:

> $\delta_t(j)$ is the probability of the most likely path ending in state j at time t, given the observations up to time t.

The Recursive Formula:

$$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot A_{ij}] \cdot B_j(x_t) \tag{1.27}$$

To retrieve the path later, we store a backpointer ψ:

$$\psi_t(j) = argmax_i [\delta_{t-1}(i) \cdot A_{ij}] \tag{1.28}$$

1.8.4 Example: The Viterbi Algorithm in Action

To illustrate the Viterbi algorithm, consider a simple Hidden Markov Model (HMM) representing a person's mood (Hidden States: *Happy*, *Sad*) based on the weather (Observations: *Sunny*, *Rainy*).

Model Parameters

Let the state space be $S = \{s_1, s_2\}$ where $s_1 =$ Happy and $s_2 =$ Sad. The transition probability matrix A and emission probability matrix B are defined as:

$$A = \begin{array}{c} \\ \text{From } H \\ \text{From } S \end{array} \begin{array}{c} \begin{array}{cc} \text{To: } H & S \end{array} \\ \begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix} \end{array}, \quad B = \begin{array}{c} \\ \text{If } H \\ \text{If } S \end{array} \begin{array}{c} \begin{array}{cc} \text{Obs: Sun} & \text{Rain} \end{array} \\ \begin{bmatrix} 0.8 & 0.2 \\ 0.1 & 0.9 \end{bmatrix} \end{array}$$

Assume an initial state distribution $\pi = [0.6, 0.4]$.

The Viterbi Recursion

Given an observation sequence $O = \{\text{Sunny}, \text{Rainy}\}$, we seek the most likely sequence of hidden states. We define $\delta_t(i)$ as the highest probability of a single path at time t that ends in state s_i:

$$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

Step 1: Initialization ($t = 1$ for "Sunny")

- $\delta_1(1) = \pi_1 \cdot b_1(\text{Sunny}) = 0.6 \cdot 0.8 = 0.48$
- $\delta_1(2) = \pi_2 \cdot b_2(\text{Sunny}) = 0.4 \cdot 0.1 = 0.04$

Step 2: Recursion ($t = 2$ for "Rainy") To find $\delta_2(1)$ (Happy at $t = 2$):

$$\delta_2(1) = \max(0.48 \cdot 0.7, 0.04 \cdot 0.4) \cdot 0.2 = \max(0.336, 0.016) \cdot 0.2 = 0.0672$$

To find $\delta_2(2)$ (Sad at $t = 2$):

$$\delta_2(2) = \max(0.48 \cdot 0.3, 0.04 \cdot 0.6) \cdot 0.9 = \max(0.144, 0.024) \cdot 0.9 = 0.1296$$

Path Termination

The most likely final state is s_2 (Sad), as $\delta_2(2) > \delta_2(1)$. By backtracking the maximum arguments (the *viterbi path*), we conclude the most probable sequence of hidden states for the observed weather is **{Happy, Sad}**.

1.8.5 Traditional HMM Inference: The Forward Algorithm

While the Viterbi algorithm finds the *best sequence*, the traditional Forward algorithm is used to calculate the **likelihood** of an observation sequence. Instead of choosing the maximum path, we sum all possible paths that could lead to a specific state.

Step-by-Step Calculation

Step 1: Initialization ($t = 1$, Obs = Sunny) Calculate the probability of starting in each state and observing "Sunny":

$$\alpha_1(1) = \pi_1 \cdot b_1(\text{Sun}) = 0.6 \cdot 0.8 = 0.48$$
$$\alpha_1(2) = \pi_2 \cdot b_2(\text{Sun}) = 0.4 \cdot 0.1 = 0.04$$

Step 2: Induction ($t = 2$, Obs = Rainy) To find the probability of being in a state at $t = 2$, we **sum** the probabilities of arriving there from all possible previous states:

$$\alpha_2(j) = \left[\sum_{i=1}^{2} \alpha_1(i) a_{ij}\right] b_j(o_2)$$

$$\begin{aligned}\alpha_2(1) &= [\alpha_1(1)a_{11} + \alpha_1(2)a_{21}] \cdot b_1(\text{Rain}) \\ &= [(0.48 \cdot 0.7) + (0.04 \cdot 0.4)] \cdot 0.2 = \mathbf{0.0704}\end{aligned}$$

$$\begin{aligned}\alpha_2(2) &= [\alpha_1(1)a_{12} + \alpha_1(2)a_{22}] \cdot b_2(\text{Rain}) \\ &= [(0.48 \cdot 0.3) + (0.04 \cdot 0.6)] \cdot 0.9 = \mathbf{0.1512}\end{aligned}$$

Chapter 2

Deep Learning fundamentals

2.1 Traditional Deep Learning

The training of deep learning models is fundamentally an optimization problem over a finite dataset. However, due to memory constraints and the geometric properties of non-convex optimization landscapes, we rarely optimize over the full dataset simultaneously. The pipeline of data ingestion—comprising the Data Loader, the Batch, and the Epoch—determines the stochastic properties of the gradient descent algorithm.

2.1.1 The Data Loader and Sampling Strategy

Let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ be a dataset consisting of N independent and identically distributed (i.i.d.) samples drawn from a data generating distribution P_{data}. The goal of the learning algorithm is to minimize the expected risk, which is approximated by the empirical risk over $\mathcal{D}$.

The **Data Loader** is an abstraction responsible for decoupling data storage from the training loop. Its primary mathematical role is to define a sampling strategy $\mathcal{S}$. In the context of Stochastic Gradient Descent (SGD), this strategy typically involves a permutation function π applied to the indices $\{1, \dots, N\}$.

$$\pi : \{1, \dots, N\} \to \{1, \dots, N\} \tag{2.1}$$

By shuffling the data at the start of every iteration sequence, we ensure that the gradient computed on any subset of data is an unbiased estimator of the true gradient of the total loss function. This stochasticity is not merely a computational convenience; it is theoretically necessary to help the optimizer escape saddle points.

2.1.2 The Mini-Batch

Processing single samples (Batch Size $B = 1$) results in noisy gradient estimates and fails to utilize the vectorization capabilities of modern GPUs. Conversely, processing the full

dataset ($B = N$, or Batch Gradient Descent) is often memory-prohibitive. We therefore utilize the **Mini-Batch**.

For a given iteration t, we draw a mini-batch $\mathcal{B}_t \subset \mathcal{D}$ with cardinality $|\mathcal{B}_t| = B$. The loss function for this batch, denoted as $\mathcal{L}_\mathcal{B}$, is the mean of the per-sample losses ℓ:

$$\mathcal{L}_\mathcal{B}(\theta) = \frac{1}{B} \sum_{k=1}^{B} \ell(f(x^{(k)}; \theta), y^{(k)}) \tag{2.2}$$

Mathematical Implications of Batch Size

The gradient of the mini-batch loss is used as an estimator for the true gradient $\nabla_\theta J(\theta)$.

$$\mathbb{E}_\mathcal{B}[\nabla_\theta \mathcal{L}_\mathcal{B}(\theta)] = \nabla_\theta J(\theta) \tag{2.3}$$

The variance of this estimator is inversely proportional to the batch size B.

- **Small B:** High variance. The "noise" in the gradient step acts as a regularizer, potentially preventing overfitting, but convergence may be unstable.
- **Large B:** Low variance. The gradient approximation is more accurate, allowing for larger learning rates, but the computational cost per step increases and the model may converge to sharp minima which generalize poorly.

2.1.3 The Epoch

An **Epoch** is defined as one complete pass through the entire dataset $\mathcal{D}$. If the batch size is B, an epoch consists of $\lceil N/B \rceil$ iterations (parameter updates).

Mathematically, the training process is a nested loop where the outer loop iterates over epochs $e \in \{1, \dots, E\}$ and the inner loop iterates over batches $\mathcal{B}$ generated by the data loader. It is standard practice to reshuffle the dataset $\mathcal{D}$ at the boundary of each epoch to prevent the model from learning the order of the data.

2.2 The Fully Connected Layer

The fundamental building block of a Deep Neural Network (DNN) is the **Fully Connected (FC)** or **Dense** layer. In terms of linear algebra, this layer performs an affine transformation mapping an input vector space to an output vector space.

2.2.1 Forward Propagation Mechanics

Let $a^{[l-1]} \in \mathbb{R}^{n^{[l-1]}}$ be the activation vector from the previous layer $l-1$. The layer l is parameterized by a weight matrix $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$ and a bias vector $b^{[l]} \in \mathbb{R}^{n^{[l]}}$.

The pre-activation (or linear hypothesis) $z^{[l]}$ is computed as:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]} \tag{2.4}$$

When processing a mini-batch of size B, the input becomes a matrix $A^{[l-1]} \in \mathbb{R}^{n^{[l-1]} \times B}$. The operation generalizes to a matrix-matrix multiplication:

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]} \tag{2.5}$$

Note that the bias vector $b^{[l]}$ is broadcasted along the columns of the resulting matrix. This operation is highly optimized in linear algebra libraries (BLAS) and serves as the computational bottleneck of the network.

2.2.2 The Necessity of Non-Linearity

A Deep Neural Network is composed of a sequence of such layers. However, an affine transformation of an affine transformation is still an affine transformation. Let $f_1(x) = W_1x + b_1$ and $f_2(x) = W_2x + b_2$. The composition is:

$$f_2(f_1(x)) = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W'x + b' \tag{2.6}$$

Without non-linear activation functions, a network of arbitrary depth collapses mathematically into a single linear layer (Generalized Linear Model). To approximate complex, non-convex functions (as guaranteed by the Universal Approximation Theorem), we must introduce non-linearities.

2.3 Activation Functions

An activation function $\sigma(\cdot)$ is applied element-wise to the pre-activation vector $z^{[l]}$ to produce the layer output $a^{[l]}$.

$$a^{[l]} = \sigma(z^{[l]}) \tag{2.7}$$

2.3.1 Sigmoid and Tanh

Historically, activation functions were inspired by the firing rates of biological neurons.

Sigmoid

The logistic sigmoid function maps inputs to the range $(0, 1)$, interpreted as a probability.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2.8}$$

The derivative is expressed conveniently in terms of the function itself:

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z)) \tag{2.9}$$

Hyperbolic Tangent (Tanh)

The Tanh function is a scaled sigmoid that maps inputs to $(-1, 1)$, which is zero-centered.

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{2.10}$$

Critique (The Vanishing Gradient Problem): Both Sigmoid and Tanh saturate. For inputs $|z| \gg 0$, the derivative approaches zero. During backpropagation, the chain rule multiplies these local gradients across layers. If the derivatives are small (< 1), the gradient signal decays exponentially as it propagates back to early layers, effectively preventing the network from learning deep representations.

2.3.2 Rectified Linear Unit (ReLU)

To address the vanishing gradient problem, the Rectified Linear Unit (ReLU) [8] has become the default activation for hidden layers.

$$\text{ReLU}(z) = \max(0, z) \tag{2.11}$$

Properties:

- **Non-saturation:** For $z > 0$, the gradient is exactly 1. This preserves the magnitude of the gradient signal through deep networks.
- **Sparsity:** For $z \leq 0$, the activation is 0. This leads to sparse representations where only a subset of neurons are active.
- **Computational Efficiency:** ReLU requires only a thresholding operation, avoiding expensive exponential calculations.

Mathematical Nuance: ReLU is not differentiable at $z = 0$. In practice, we use the sub-gradient convention, setting the derivative to 0 or 1 at $z = 0$ without adverse effects on optimization.

2.4 Structural Components

Beyond dense layers and activations, modern architectures employ structural layers to manage dimensionality (Pooling) and distribution statistics (Normalization).

2.4.1 Pooling Layers

Pooling layers reduce the spatial dimensions of the input volume (down-sampling). This serves two purposes: reducing the number of parameters and computation, and inducing **translation invariance**.

Max Pooling

Given an input matrix (or tensor) X, Max Pooling applies a window function (kernel) over the input and outputs the maximum value within that window. For a region $R_{i,j}$ defined by the filter size and stride, the output $y_{i,j}$ is:

$$y_{i,j} = \max_{(p,q)\in R_{i,j}} x_{p,q} \tag{2.12}$$

Crucially, pooling layers generally have no learnable parameters. The gradient during backpropagation is routed only to the input neuron that had the maximum value (the "winner"), while other gradients are set to zero.

2.4.2 Batch Normalization

Training deep networks is complicated by **Internal Covariate Shift**—the phenomenon where the distribution of layer inputs changes during training as the parameters of previous layers change. This forces layers to continuously adapt to new input distributions.

Batch Normalization (Batch Norm) addresses this by re-centering and re-scaling the inputs based on mini-batch statistics.

The Algorithm

Consider a mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$. The Batch Norm transformation is applied as follows:

1. **Calculate Batch Mean:**
$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{2.13}$$
2. **Calculate Batch Variance:**
$$\sigma^2_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \tag{2.14}$$
3. **Normalize:**
$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \tag{2.15}$$
Here, ϵ is a small constant for numerical stability.
4. **Scale and Shift (Learnable):** Simply normalizing inputs to have zero mean and unit variance limits the representation power (e.g., it constrains inputs to the linear regime of a sigmoid). We introduce learnable parameters γ (scale) and β (shift):
$$y_i = \gamma\hat{x}_i + \beta \tag{2.16}$$

During **inference**, the batch statistics are unavailable. Instead, the layer uses a running average of the mean and variance computed during the training phase.

2.4.3 Dropout: Stochastic Regularization

Deep Neural Networks with millions of parameters are prone to **Overfitting**, where the model memorizes noise in the training data rather than learning generalizable features. **Dropout** [9] is a regularization technique that prevents complex co-adaptations of neurons.

The Mathematical Mechanism

Dropout temporarily "drops" (zeros out) a random set of neurons during the training phase. Let h be the output vector of a layer. We define a binary mask vector r drawn from a Bernoulli distribution with parameter p (the probability of *dropping* a unit, typically $p = 0.5$).

$$r_j \sim \text{Bernoulli}(1 - p) \tag{2.17}$$

The modified output $\tilde{h}$ is computed as the element-wise product:

$$\tilde{h} = h \odot r \tag{2.18}$$

Inverted Dropout and Inference

A crucial mathematical detail is the discrepancy between training and testing energy. During training, the expected output of a neuron is reduced by a factor of $(1 - p)$. To maintain the same expected magnitude during inference (when dropout is turned off), we must scale the activations.

Modern frameworks utilize **Inverted Dropout**, which performs the scaling during the *training* phase rather than test phase. This simplifies deployment.

$$\tilde{h}_{train} = \frac{1}{1 - p}(h \odot r) \tag{2.19}$$

By dividing by $(1 - p)$ during training, the expected value $\mathbb{E}[\tilde{h}_{train}] = h$, ensuring that the test-time operation (which is simply identity) matches the training statistics without modification.

2.5 Loss Functions

The loss function (or cost function) $\mathcal{L}$ quantifies the discrepancy between the model prediction $\hat{y}$ and the ground truth y. It defines the manifold over which the optimizer navigates.

2.5.1 Maximum Likelihood Estimation (MLE)

Most deep learning loss functions are derived from the principle of Maximum Likelihood. We seek the parameters θ that maximize the probability of the observed data.

$$\theta_{\text{MLE}} = \arg\max_{\theta} \sum_{i=1}^{N} \log P(y^{(i)}|x^{(i)};\theta) \tag{2.20}$$

Minimizing the negative log-likelihood (NLL) is equivalent to maximizing the likelihood.

2.5.2 Cross-Entropy Loss (Classification)

For multi-class classification problems with C classes, the network outputs a probability distribution vector $\hat{y}$ (typically via the Softmax function) and the target y is a one-hot encoded vector.

The Cross-Entropy loss is defined as:

$$\mathcal{L}_{CE}(y,\hat{y}) = -\sum_{k=1}^{C} y_k \log(\hat{y}_k) \tag{2.21}$$

Since y is a one-hot vector where only the entry corresponding to the true class c is 1, this simplifies to:

$$\mathcal{L}_{CE} = -\log(\hat{y}_c) \tag{2.22}$$

Connection to KL Divergence

Minimizing Cross-Entropy is mathematically equivalent to minimizing the Kullback-Leibler (KL) Divergence between the empirical data distribution P and the model distribution Q.

$$D_{KL}(P||Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)} = \mathbb{E}_{x\sim P}[\log P(x) - \log Q(x)] \tag{2.23}$$

Since the entropy of the empirical data $P(x)$ is constant with respect to the model parameters, minimizing KL divergence is identical to minimizing the Cross-Entropy.

2.5.3 Mean Squared Error (Regression)

For regression tasks where the target $y \in \mathbb{R}$, we typically assume the data follows a Gaussian distribution with constant variance.

$$p(y|x) = \mathcal{N}(y;\hat{y}(x;\theta),\sigma^2) \tag{2.24}$$

Maximizing the log-likelihood of this Gaussian yields the Mean Squared Error (MSE) objective:

$$\mathcal{L}_{MSE}(y,\hat{y}) = \frac{1}{N}\sum_{i=1}^{N} \|y^{(i)} - \hat{y}^{(i)}\|_2^2 \tag{2.25}$$

The gradient of the MSE is linear: $\nabla_{\hat{y}}\mathcal{L} \propto (\hat{y} - y)$. This ensures that large errors result in large gradients, driving the model quickly toward the solution, though it also makes the loss sensitive to outliers.

2.6 Convolutional Neural Networks (CNNs)

While Fully Connected layers are universal approximators, they ignore the topological structure of the input data (e.g., spatial locality in images). Convolutional Neural Networks (CNNs) [10] introduce **inductive biases**—translation invariance and locality—to process grid-structured data efficiently.

2.6.1 The Convolution Operation

In the context of deep learning, the "convolution" operation is technically a **Cross-Correlation**. We slide a learnable filter (or kernel) over the input to produce a feature map.

Mathematical Definition

Let the input be a 2D image I of size $H \times W$ and the kernel be K of size $k_h \times k_w$. The output feature map S at position (i, j) is computed as the Frobenius inner product of the kernel and the local input patch:

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I(i+m, j+n)K(m,n) \tag{2.26}$$

Note on Strict Convolution: In pure signal processing, a convolution requires flipping the kernel relative to the input ($K(i-m, j-n)$). In deep learning, since the kernel weights are learned, this flipping is unnecessary and omitted for implementation efficiency.

Tensor Operations

Real-world CNN layers operate on 3D tensors. If the input has C_{in} channels (e.g., RGB implies $C_{in} = 3$), the kernel must also have depth C_{in}. If we desire C_{out} output feature maps, we utilize C_{out} distinct kernels. The weights W have the shape $(C_{out}, C_{in}, k_h, k_w)$.

2.6.2 Spatial Control: Padding and Stride

Two hyperparameters control the spatial dimensions of the output volume.

1. **Stride (S):** The step size of the sliding window. A stride $S > 1$ results in down-sampling the input.
2. **Padding (P):** Adding zero-valued pixels around the border of the input. This preserves spatial dimensions and allows the kernel to process edge pixels.

Output Dimension Formula: Given an input dimension W_{in}, kernel size K, padding P, and stride S, the output dimension W_{out} is:

$$W_{out} = \left\lfloor \frac{W_{in} - K + 2P}{S} \right\rfloor + 1 \tag{2.27}$$

2.6.3 The CNN Block Pipeline

A single convolution is rarely used in isolation. It forms a functional unit, typically followed immediately by non-linearities and structural layers discussed in previous sections.

The Classical Block

In architectures like AlexNet or VGG, the standard repeated motif is:

$$\texttt{Conv} \rightarrow \texttt{Batch Norm} \rightarrow \texttt{ReLU} \rightarrow \texttt{Pooling}$$

Here, the Convolution extracts linear features, Batch Norm stabilizes the distribution, ReLU introduces non-linearity, and Pooling reduces the dimension to increase the **Receptive Field** of subsequent layers.

2.6.4 Architectural Case Studies

The evolution of CNNs is best understood through standout architectures that introduced paradigm shifts.

AlexNet (2012[11]: The Deep Learning Spark

AlexNet was the pivotal model that demonstrated the viability of Deep Learning on the ImageNet dataset.

- **Innovation:** It was one of the first to successfully combine deep stacking of convolutional layers (5 Conv layers) with **ReLU** activations (solving the vanishing gradient problem) and **Dropout** (preventing overfitting in large FC layers).
- **Legacy:** It established the standard pattern of increasing channel depth ($C \uparrow$) while decreasing spatial resolution ($H, W \downarrow$) as the data flows deeper into the network.

U-Net (2015)[12]: Segmentation and Skip Connections

While AlexNet focuses on classification (image → label), U-Net was designed for biomedical image segmentation (image → mask).

- **Structure:** It features an Encoder (contracting path) and a Decoder (expanding path), giving it a symmetric "U" shape.
- **Skip Connections:** The critical innovation is the use of long-range skip connections that **concatenate** high-resolution feature maps from the encoder directly to the up-sampled features in the decoder.

$$x_{\text{decoder}} = \text{Concat}(Up(x_{\text{prev}}), x_{\text{encoder}}) \tag{2.28}$$

  This allows the network to localize features precisely by retaining high-frequency spatial details that are typically lost during pooling.

ConvNeXt (2022)[13]: The Modernization

In the 2020s, Vision Transformers (ViTs) began outperforming CNNs. ConvNeXt is a "pure" CNN architecture designed to modernize the convolution operation to compete with Transformers.

- **Patchify:** It uses a large stride convolution (e.g., 4×4, stride 4) to mimic the "patch" tokenization of transformers.
- **Depthwise Separable Convolutions:** It splits convolution into spatial filtering (Depthwise) and channel mixing (Pointwise 1×1 Conv), reducing computational cost significantly.
- **Large Kernels:** Unlike the standard 3×3 kernels of the past decade, ConvNeXt employs large kernels (e.g., 7×7) to capture a global context similar to the Self-Attention mechanism.

2.7 Object Detection and Advanced Architectures

While image classification answers the question "What is in this image?", it fails to address "Where is it?". **Object Detection** is the dual task of classification (predicting class labels) and localization (predicting bounding box coordinates). This introduces significant complexity: the output dimensionality changes based on the number of objects, and the model must handle objects of varying scales and aspect ratios.

2.7.1 The Region-Based Paradigm (R-CNN)

Before 2014, object detection relied on sliding windows—passing a classifier over every possible sub-window of an image. This is computationally intractable ($O(N^2)$ complexity) given the high cost of Deep CNNs.

R-CNN: Regions with CNN Features

The **R-CNN** (Region-based Convolutional Neural Network) [14, 15]architecture proposed a pivotal shift: instead of processing every pixel, we only process "Region Proposals."

The Pipeline:

1. **Region Proposal:** An external algorithm (e.g., Selective Search) generates ≈ 2000 candidate regions (RoIs) based on color, texture, and shape hierarchies.
2. **Warping:** Since CNNs require fixed-size inputs (e.g., 224×224), each arbitrary rectangular proposal is warped (resized) to the required dimension.
3. **Feature Extraction:** A standard CNN (e.g., AlexNet) extracts a feature vector (4096-d) from the warped region.
4. **Classification and Regression:** Support Vector Machines (SVMs) classify the feature vector, and a linear regressor refines the bounding box coordinates.

Mathematical Bottleneck: R-CNN is slow because it performs the forward pass of the CNN 2000 times per image. It fails to share computation.

$$\text{Cost} \propto N_{\text{proposals}} \times \text{Cost}_{\text{CNN}} \tag{2.29}$$

Fast R-CNN and RoI Pooling

Fast R-CNN solved the speed bottleneck by running the CNN *once* on the entire image to generate a global Feature Map. The challenge then becomes: how do we extract fixed-length vectors from variable-sized regions of the feature map?

The Solution: Region of Interest (RoI) Pooling. Let the feature map have dimensions $H \times W$. An RoI is a window (r, c, h, w). We want to pool this into a fixed grid of size $H' \times W'$ (e.g., 7×7). We divide the RoI into an $H' \times W'$ grid of sub-windows, each of approximate size $h/H' \times w/W'$. Max pooling is applied to each sub-window.

$$y_{i,j} = \max_{p \in \text{bin}(i,j)} x_p \tag{2.30}$$

This operation is differentiable (sub-gradients are routed to the argmax index), allowing end-to-end training.

Faster R-CNN: The Region Proposal Network (RPN)

Selective Search (used in R-CNN) is a slow, CPU-bound algorithm. Faster R-CNN replaces it with a fully convolutional **Region Proposal Network (RPN)**.

Anchor Boxes: The RPN slides a small window over the feature map. At each location, it predicts offsets relative to k reference boxes called **Anchors** (of different scales and

aspect ratios). If the feature map size is $W \times H$, the RPN outputs $W \times H \times k$ proposals simultaneously.

The Loss Function: The RPN minimizes a multi-task loss:

$$L(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*) \tag{2.31}$$

Where:

- p_i: Predicted probability of anchor i being an object.
- t_i: Predicted coordinates offsets.
- L_{cls}: Log loss over two classes (object vs. background).
- L_{reg}: Smooth L1 loss for bounding box regression, active only if the anchor contains an object ($p_i^* = 1$).

2.7.2 Feature Pyramid Networks (FPN): Bridging Semantics and Resolution

A fundamental conflict exists in standard CNN architectures (like AlexNet or ResNet) regarding the trade-off between **semantics** and **resolution**.

- **Deep Layers (e.g., Conv5):** Have high semantic value (robust to lighting, pose, class variations) but low spatial resolution. They are excellent for classification but struggle to localize small objects.
- **Shallow Layers (e.g., Conv2):** Have high spatial resolution but low semantic value (detecting edges, curves). They are good for localization but cannot differentiate between complex classes.

To solve this, **Feature Pyramid Networks (FPN)** [16] was explored. This architecture effectively bridges the gap between the standard feedforward CNN and the symmetric Encoder-Decoder structure seen in U-Net.

The Structural Bridge: FPN vs. U-Net

Both U-Net and FPN utilize a symmetric shape consisting of a contracting path (encoder) and an expanding path (decoder) with skip connections. However, their goals differ mathematically:

- **U-Net (Segmentation):** The goal is to reconstruct a *single* high-resolution output mask. It concatenates features from the encoder to the decoder to recover lost spatial details, culminating in one final prediction layer.

- **FPN (Detection):** The goal is to detect objects at *multiple scales*. Instead of producing a single output, FPN treats every level of the decoder pyramid as a predictive layer. It creates a hierarchy of feature maps where *every* level contains strong semantic information.

The FPN Algorithm

The FPN construction involves three distinct pathways:

1. **Bottom-Up Pathway (The Encoder):** This is the standard convolutional backbone (e.g., ResNet-50). It computes a feature hierarchy with a scaling step of 2. Let $\{C_2, C_3, C_4, C_5\}$ denote the output feature maps of the last residual block at each stage (where C_2 has the highest resolution).
2. **Top-Down Pathway (The Decoder):** We wish to hallucinate high-resolution features that are also semantically strong. We do this by upsampling the spatially coarser feature maps from higher pyramid levels.
3. **Lateral Connections (The Bridge):** Merely upsampling deep features is insufficient because the precise location of objects is lost. We recover this by merging the upsampled map with the corresponding bottom-up map C_i via a **Lateral Connection**.

   Mathematically, for a pyramid level i:

$$P_i = \underbrace{\text{Conv}_{1\times1}(C_i)}_{\text{Lateral Project}} + \underbrace{\text{Upsample}_{\times2}(P_{i+1})}_{\text{Top-Down}} \tag{2.32}$$

   - **$\text{Conv}_{1\times1}$:** Reduces the channel dimensions of the raw backbone features C_i (which may have 2048 channels) to a fixed dimension d (typically 256), forcing the network to compress information.
   - **Addition:** Unlike U-Net which often uses concatenation, FPN typically uses element-wise addition to fuse the signals.

The Multi-Scale Output

The result is a new pyramid $\{P_2, P_3, P_4, P_5\}$. Unlike standard classification networks that put a classifier only at the end (P_5), FPN places a detector (RPN or classifier head) at **every level** of this pyramid.

- Large objects are detected on P_5 (deep, coarse).
- Small objects are detected on P_2 (shallow, fine).

This "divide and conquer" strategy ensures that small objects—which would otherwise vanish in the low resolution of C_5—are detected on P_2, but with the semantic benefit of the top-down context P_5 added in.

2.7.3 Single-Shot Detection (YOLO)

The R-CNN family is a **Two-Stage** approach: (1) Generate Proposals, (2) Classify Proposals. While accurate, it is computationally heavy. **YOLO (You Only Look Once)** reframes object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities.

The Grid Concept

YOLO divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that cell is responsible for detecting it. Each cell predicts B bounding boxes and C class probabilities. The output tensor size is $S \times S \times (B \times 5 + C)$. The "5" corresponds to: t_x, t_y, t_w, t_h (coordinates) and p_o (confidence score).

YOLO Loss and IoU

YOLO uses a specialized Sum-Squared Error loss. A critical component of training (and evaluation) is the **Intersection over Union (IoU)** metric. Given a predicted box B_p and a ground truth box B_{gt}:

$$\text{IoU}(B_p, B_{gt}) = \frac{\text{Area}(B_p \cap B_{gt})}{\text{Area}(B_p \cup B_{gt})} \tag{2.33}$$

The confidence score p_o reflects $\Pr(\text{Object}) \times \text{IoU}^{\text{truth}}_{\text{pred}}$.

Non-Maximum Suppression (NMS): At inference, YOLO outputs thousands of overlapping boxes. NMS filters these by:

1. Discarding boxes with confidence below a threshold.
2. Selecting the box with the highest confidence as the "winner".
3. Removing any remaining boxes that have high IoU (> 0.5) with the winner.

2.7.4 Vision Transformers (ViT)

By 2020, the dominance of CNNs was challenged by the Transformer architecture, originally designed for Natural Language Processing (NLP). The **Vision Transformer (ViT)** treats an image not as a grid of pixels, but as a sequence of patches.

Patch Embedding

An image $x \in \mathbb{R}^{H \times W \times C}$ is reshaped into a sequence of flattened 2D patches $x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$, where (P, P) is the resolution of each patch and $N = HW/P^2$ is the number

of patches (sequence length). These patches are linearly projected to a latent vector size D:

$$z_0 = [x_{\text{class}}; x_p^1 E; x_p^2 E; \ldots; x_p^N E] + E_{\text{pos}} \tag{2.34}$$

- E: Learnable linear projection (equivalent to a convolution with stride P).
- E_{pos}: Learnable position embeddings added to the patches to retain spatial information (since Transformers are permutation invariant).
- x_{class}: A learnable "CLS token" prepended to the sequence. Its state at the final layer serves as the image representation.

Self-Attention Mechanism

The core of ViT is the Multi-Head Self-Attention (MSA) layer, which replaces the convolution. Given Queries Q, Keys K, and Values V derived from the input embeddings:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.35}$$

Comparison to CNNs:

- **Global Receptive Field:** A CNN layer only sees local neighbors (3×3). An Attention layer allows every patch to attend to every other patch immediately in the first layer. This models long-range dependencies efficiently.
- **Inductive Bias:** CNNs have hard-coded biases for translation invariance and locality. ViTs lack these biases; they must learn them from data. Consequently, ViTs require significantly more training data (or strong regularization) to generalize well, but they scale better with massive datasets (JFT-300M) where CNN performance saturates.

2.8 Sequence Modeling: Recurrent Neural Networks

While Convolutional Networks excel at processing spatial grids (images), they struggle with sequential data where the "context" depends on previous inputs (e.g., time-series forecasting, natural language processing). To model temporal dependencies, the **Recurrent Neural Network (RNN)** [17] was introduced.

2.8.1 The Recurrent Mechanism

A standard feedforward network maps input $x \to y$. An RNN maps a sequence $[x^{(1)}, \ldots, x^{(T)}]$ to a sequence $[y^{(1)}, \ldots, y^{(T)}]$ by maintaining an internal **Hidden State** $h^{(t)}$ that acts as a memory of the past.

Mathematical Formulation

At time step t, the hidden state $h^{(t)}$ is updated based on the current input $x^{(t)}$ and the previous hidden state $h^{(t-1)}$.

$$h^{(t)} = \sigma(W_{hh}h^{(t-1)} + W_{xh}x^{(t)} + b_h) \tag{2.36}$$

The output $y^{(t)}$ is then derived from this hidden state:

$$y^{(t)} = W_{hy}h^{(t)} + b_y \tag{2.37}$$

Crucially, the weight matrices W_{hh}, W_{xh}, W_{hy} are **shared** across all time steps. This allows the model to process sequences of arbitrary length using a fixed number of parameters.

2.8.2 Backpropagation Through Time (BPTT) and Vanishing Gradients

To train an RNN, we unroll the graph over time and apply the chain rule. This algorithm is known as **Backpropagation Through Time (BPTT)**.

The gradient of the total loss $\mathcal{L}$ with respect to the recurrent weights W_{hh} involves a product of Jacobians:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(t)}}{\partial W_{hh}} \quad \text{where} \quad \frac{\partial h^{(t)}}{\partial h^{(k)}} = \prod_{i=k+1}^{t} \frac{\partial h^{(i)}}{\partial h^{(i-1)}} \tag{2.38}$$

If the largest eigenvalue of the weight matrix W_{hh} is less than 1 (which is common to prevent explosion), this product decays exponentially to zero as the time gap $t-k$ increases. **Consequence:** The Simple RNN forgets information from the distant past. It cannot learn long-term dependencies (e.g., the subject of a sentence appearing 20 words before the verb).

2.8.3 Long Short-Term Memory (LSTM)

To solve the vanishing gradient problem, the **Long Short-Term Memory (LSTM)**[18] architecture introduces a dedicated memory highway known as the **Cell State** ($c^{(t)}$).

The Cell State: The Superhighway

Unlike the hidden state in a simple RNN which is constantly rewritten by non-linearities (tanh), the Cell State $c^{(t)}$ flows through time with only minor linear interactions. This allows gradients to flow backwards unchanged, preserving long-term memory.

The LSTM controls the flow of information using three binary-like **Gates** (values in $[0, 1]$ via sigmoid σ).

LSTM Equations

1. **Forget Gate ($f^{(t)}$):** Decides what to throw away from the old cell state.
$$f^{(t)} = \sigma(W_f \cdot [h^{(t-1)}, x^{(t)}] + b_f) \tag{2.39}$$
2. **Input Gate ($i^{(t)}$) and Candidate Memory ($\tilde{c}^{(t)}$):** Decides what new information to store.
$$i^{(t)} = \sigma(W_i \cdot [h^{(t-1)}, x^{(t)}] + b_i) \tag{2.40}$$
$$\tilde{c}^{(t)} = \tanh(W_c \cdot [h^{(t-1)}, x^{(t)}] + b_c) \tag{2.41}$$
3. **Cell State Update:** The core operation. The old state is scaled by the forget gate, and the new candidate is scaled by the input gate.
$$c^{(t)} = \underbrace{f^{(t)} \odot c^{(t-1)}}_{\text{Forget Old}} + \underbrace{i^{(t)} \odot \tilde{c}^{(t)}}_{\text{Add New}} \tag{2.42}$$
4. **Output Gate ($o^{(t)}$):** Decides what part of the cell state to expose as the hidden state.
$$o^{(t)} = \sigma(W_o \cdot [h^{(t-1)}, x^{(t)}] + b_o) \tag{2.43}$$
$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)}) \tag{2.44}$$

2.8.4 Gated Recurrent Unit (GRU)

The **Gated Recurrent Unit (GRU)** [19] is a simplified variant of the LSTM. It was designed to capture the same long-term dependencies but with a more streamlined architecture.

GRU Equations

The GRU merges the Cell State and Hidden State into a single vector $h^{(t)}$. It also reduces the number of gates from three to two:

1. **Reset Gate ($r^{(t)}$):** Determines how much of the past knowledge to forget.
2. **Update Gate ($z^{(t)}$):** A combination of the LSTM's forget and input gates. It balances how much of the previous state to keep vs. the new candidate.

$$h^{(t)} = (1 - z^{(t)}) \odot h^{(t-1)} + z^{(t)} \odot \tilde{h}^{(t)} \tag{2.45}$$

2.8.5 Comparative Analysis: LSTM vs. GRU

A frequent question in deep learning architecture design is the choice between LSTM and GRU. While GRU is newer, the "older" LSTM offers distinct theoretical advantages regarding memory separation.

The Memory Isolation Issue

The primary structural difference lies in the **Cell State**.

- **LSTM (Separated Memory):** Maintains two vectors: $c^{(t)}$ (Cell State) and $h^{(t)}$ (Hidden State). The Cell State can theoretically carry information for infinite time steps without modification, while the Hidden State represents the "current view" of that memory used for immediate predictions.
- **GRU (Coupled Memory):** Exposes the full memory content $h^{(t)}$ to other units in the network. There is no protected "cell" that is hidden from the output.

Implication: The LSTM's separation allows it to maintain a precise long-term counter (in $c^{(t)}$) that does not disturb the immediate output. In complex tasks requiring very long-term memory (e.g., nested logic in code generation), LSTMs often outperform GRUs because the GRU's memory is constantly "exposed" and potentially diluted by the need to produce immediate outputs.

Parameter and Size Comparison

Let N be the input dimension and H be the hidden dimension. The parameter cost is determined by the linear layers in the gates.

- **LSTM:** Has 4 distinct neural network layers (Forget, Input, Output, Candidate).

$$P_{\text{LSTM}} \approx 4 \times (H^2 + NH + H) \tag{2.46}$$

- **GRU:** Has 3 distinct neural network layers (Reset, Update, Candidate).

$$P_{\text{GRU}} \approx 3 \times (H^2 + NH + H) \tag{2.47}$$

Summary: The GRU has approximately **25% fewer parameters** than the LSTM. This makes the GRU faster to train and less prone to overfitting on smaller datasets. However, for large-scale tasks with abundant data, the LSTM's additional expressivity and memory protection often justify the extra computational cost.

2.9 The Transformer and Attention Mechanisms

By 2017, the field of Deep Learning reached an inflection point. While RNNs and LSTMs theoretically solved sequence modeling, they suffered from a fatal flaw: sequential processing precludes parallelization. Furthermore, despite the LSTM's gating, compressing a long source sentence into a single fixed-length context vector created an informational bottleneck.

The solution was to dispense with recurrence entirely and rely on a mechanism that allows the model to access any part of the sequence history directly. This paradigm is the **Transformer** [20].

2.9.1 The Motivation: The Information Bottleneck

Consider a Sequence-to-Sequence (Seq2Seq) task like translation. An RNN Encoder processes the input $x_1, \ldots, x_T$ and produces a final hidden state h_T.

$$\text{Context} = h_T \tag{2.48}$$

The Decoder must generate the entire output translation based solely on this single vector h_T. For long sentences (e.g., 50+ words), h_T acts as a "bottleneck," forcing the model to lose fine-grained details. **Attention** solves this by allowing the Decoder to "look back" at *all* encoder hidden states $\{h_1, \ldots, h_T\}$ dynamically at every decoding step.

2.9.2 The Anatomy of Attention

The modern definition of Attention abstracts the concept into a retrieval system based on **Queries (Q)**, **Keys (K)**, and **Values (V)**.

The Retrieval Analogy

Imagine a database (a dictionary in Python).

- **Key (K):** The label associated with the data (e.g., "filename").
- **Value (V):** The actual content of the data (e.g., the file bytes).
- **Query (Q):** The search term you are looking for.

In a harsh database, you only get the value if $Query == Key$. In the differentiable world of Deep Learning, we compute a *soft* match. We calculate how similar the Query is to the Key, and retrieve a weighted average of the Values.

2.9.3 Scaled Dot-Product Attention

The core mathematical operation of the Transformer is Scaled Dot-Product Attention. Let $Q \in \mathbb{R}^{N \times d_k}$, $K \in \mathbb{R}^{M \times d_k}$, and $V \in \mathbb{R}^{M \times d_v}$.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.49}$$

Why This Formula "Focuses" Attention

To understand why this helps the model attend to specific data, we must break down the operations:

1. **The Similarity Score (QK^T):** The dot product between two vectors q_i and k_j measures their geometric alignment.

   - If q_i and k_j are pointing in the same direction, the product is large (positive).
   - If they are orthogonal (unrelated), the product is zero.

   Therefore, the matrix product QK^T generates a **Score Matrix** of size $N \times M$, representing the raw compatibility between every query and every key.

2. **The Scaling Factor ($\frac{1}{\sqrt{d_k}}$):** For large dimensions d_k, the dot products can grow very large in magnitude. This pushes the subsequent Softmax function into regions where gradients are extremely small (vanishing gradients). Scaling by $\sqrt{d_k}$ normalizes the variance to 1, ensuring stable gradients.

3. **The Softmax Filter:** The Softmax function converts raw scores into a probability distribution (summing to 1).

   $$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})} \tag{2.50}$$

   If the score for word A is high and word B is low, the Softmax yields weights like $[0.95, 0.05]$.

4. **Weighted Aggregation ($\cdot V$):** Finally, we multiply these weights by the Values V.

   $$\text{Output}_i = 0.95 v_A + 0.05 v_B \tag{2.51}$$

   Result: The output vector is dominated by the content of v_A. The model has effectively "attended" to input A and ignored input B, solely based on the vector similarity between the Query and the Key.

2.9.4 Multi-Head Attention

A single attention mechanism can only focus on one aspect of the relationship (e.g., syntactic agreement). To capture multiple types of relationships simultaneously (e.g., semantic vs. grammatical), we use **Multi-Head Attention**.

We project Q, K, V into h different subspaces using learnable linear projections W_i^Q, W_i^K, W_i^V.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{2.52}$$

The results are concatenated and projected back:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2.53}$$

This allows one head to attend to "who is doing the action" while another attends to "when the action happened."

2.9.5 Self-Attention vs. Cross-Attention

The Transformer utilizes this mechanism in two distinct ways:

1. **Self-Attention:** Here, $Q = K = V = X$ (the input sequence). Every token in the sequence attends to every other token in the same sequence. *Purpose:* Contextualization. For the word "bank" in "river bank," self-attention looks at the word "river" to adjust the representation of "bank" away from the financial meaning.
2. **Cross-Attention (Encoder-Decoder Attention):** Here, Queries Q come from the Decoder (the target sentence being generated), while Keys K and Values V come from the Encoder (the source sentence). *Purpose:* Alignment. It allows the decoder to find the relevant information in the source sentence to generate the next word.

2.9.6 Positional Encoding: Injecting Order

Unlike Recurrent Neural Networks (RNNs) which process tokens sequentially ($t_1 \to t_2 \to \dots$), the Transformer architecture is **permutation invariant**. The self-attention mechanism treats the input as a "bag of words" (a set); if you shuffle the input words, the attention outputs are identical, merely shuffled.

To enable the model to distinguish "The dog bit the man" from "The man bit the dog," we must inject information about the relative or absolute position of the tokens in the sequence. This is achieved via **Positional Encodings (PE)**.

Mathematical Formulation (Sinusoidal)

In the original Transformer, the positional encoding is a fixed, non-learnable vector added element-wise to the input embedding. Let pos be the position in the sequence and i be the dimension index within the embedding vector of size d_{model}.

The encoding is defined using a geometric progression of wavelengths, controlled by a timescale base hyperparameter τ (typically set to 10000).

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{\tau^{2i/d_{\text{model}}}}\right) \tag{2.54}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{\tau^{2i/d_{\text{model}}}}\right) \tag{2.55}$$

The Role of the Base τ

The constant τ (standardized as 10000.0) is not arbitrary; it controls the range of wavelengths across the embedding dimensions.

- **Wavelength Distribution:** The wavelength λ_i for dimension i is $2\pi \cdot \tau^{\frac{2i}{d_{\text{model}}}}$.
- **Short vs. Long Term:**
  - At $i = 0$ (low dimension), the wavelength is short (2π), capturing local variations.
  - At $i = d/2$ (high dimension), the wavelength is large ($2\pi \cdot \tau$), capturing global position.
- **Avoiding Aliasing:** We choose τ such that the longest wavelength is larger than the maximum expected sequence length. If τ is too small (e.g., 100) and the sequence is 500 words long, the pattern will repeat, making position 5 indistinguishable from position 105.

Tuning τ: In modern Long-Context models (e.g., processing books or genomic sequences), τ is often increased (e.g., to 10^6 or $5 \cdot 10^5$) to ensure uniqueness across millions of tokens.

Why Sinusoids? (The Rotation Property)

This function was chosen because it allows the model to easily learn to attend by relative positions. For any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

$$\begin{bmatrix} \sin(pos+k) \\ \cos(pos+k) \end{bmatrix} = \begin{bmatrix} \cos(k) & \sin(k) \\ -\sin(k) & \cos(k) \end{bmatrix} \begin{bmatrix} \sin(pos) \\ \cos(pos) \end{bmatrix} \tag{2.56}$$

This rotation matrix property implies that the model can learn relationships like "the word 3 positions back" regardless of the absolute position in the sentence.

Forming PE in Real Systems

In a practical deep learning system (e.g., PyTorch or TensorFlow), we do not compute these sines and cosines on the fly for every batch due to computational overhead. Instead, we compute a **cached matrix**.

Implementation Steps:

1. **Define Max Length:** We select a maximum sequence length (e.g., `max_len = 5000`) that the model will ever encounter.
2. **Construct the Matrix:** We create a constant matrix $PE \in \mathbb{R}^{\text{max\_len} \times d_{\text{model}}}$.
   - The rows correspond to positions $0, 1, \ldots, \text{max\_len} - 1$.
   - The columns correspond to the sinusoidal frequencies.
3. **The Division Term:** To implement the τ base numerically stable, we compute in log-space:

$$\text{div\_term} = \exp\left(\text{arange}(0, d_{\text{model}}, 2) \cdot \frac{-\log(\tau)}{d_{\text{model}}}\right) \tag{2.57}$$

4. **Injection (Forward Pass):** Given an input batch of embeddings $X \in \mathbb{R}^{B \times L \times d_{\text{model}}}$ (where L is the current sequence length):

$$X_{\text{encoded}} = X + PE[0 : L, :] \tag{2.58}$$

Crucially, gradients are **not** calculated for the PE matrix; it is a fixed buffer, not a learnable parameter.

2.9.7 Learnable Positional Embeddings

While the sinusoidal formulation is elegant and parameter-free, many dominant architectures (including BERT, GPT-2, and ViT) utilize **Learnable Positional Embeddings** instead.

In this approach, position is treated as a discrete variable, similar to a token ID. We define a learnable weight matrix $W_{pos} \in \mathbb{R}^{L_{max} \times d_{\text{model}}}$. Given a position index t, the encoding is simply a lookup operation:

$$PE_{(t)} = W_{pos}[t] \tag{2.59}$$

Comparison:

- **Flexibility:** Learnable embeddings allow the model to discover arbitrary positional relationships that may not be captured by a rigid sinusoidal frequency.
- **The Length Limit:** A major drawback is the lack of extrapolation. If a model is trained with $L_{max} = 512$, the weight matrix has exactly 512 rows. Processing a sequence of length 513 is impossible without resizing and re-finetuning the model (or using interpolation techniques).

Why "Dense"? Occasionally, literature refers to this as a "dense" encoding because the lookup operation is mathematically equivalent to applying a one-hot vector of the position to a Dense (Fully Connected) layer without bias.

$$PE_{(t)} = \text{OneHot}(t) \cdot W_{dense} \tag{2.60}$$

2.9.8 Positional Encodings in Modern Applications

While the fixed sinusoidal encoding was the foundation, modern architectures have evolved to suit specific domain constraints.

Natural Language Processing (NLP)

In state-of-the-art Large Language Models (LLMs) like LLaMA and PaLM, absolute positional encodings have largely been replaced.

- **Rotary Positional Embeddings (RoPE):** Instead of *adding* the position vector to the embedding, RoPE *multiplies* the context-dependent query and key vectors by a rotation matrix derived from the position (leveraging the rotation property explicitly). This strictly enforces relative position dependencies, allowing models to generalize better to sequence lengths longer than those seen during training (extrapolation).

Human Activity Recognition (HAR)

HAR involves analyzing time-series data from wearable sensors (accelerometers, gyroscopes). The data is continuous and numerical, unlike discrete words in NLP.

- **The Challenge:** A sensor reading at $t = 100$ms is often highly correlated with $t = 101$ms. Standard Transformers treat these time steps as independent patches.
- **Usage:** In architectures like the *Limb-based Transformer*, Positional Encoding is critical to define the **temporal order** of sensor frames. Without PE, the model cannot distinguish between "lifting an arm" (sequence $A \to B \to C$) and "lowering an arm" (sequence $C \to B \to A$), as the set of sensor values is identical.
- **Learnable Encodings:** Unlike NLP which often uses fixed sinusoids, HAR models frequently use **Learnable Positional Embeddings**. Since the "grammar" of human movement is less structured than language, allowing the model to learn optimal position representations for sensor windows often yields higher accuracy.

Computer Vision (ViT)

In Vision Transformers (ViT), an image is chopped into patches (e.g., 16×16 pixels).

- **2D Awareness:** While standard PE is 1D (sequence order), images are 2D. However, standard ViT simply flattens the grid $(H/P \times W/P) \to N$ and uses learnable 1D embeddings.
- **Observation:** Interestingly, visualization of these learned embeddings shows that the model reconstructs 2D topology on its own; the embedding for patch i is mathematically similar to the embeddings of its spatial neighbors (up, down, left, right), effectively relearning the grid structure.

2.10 Large Language Models and Multimodality

The Transformer architecture, originally designed for translation, evolved into the foundation of Large Language Models (LLMs). These models do not merely process text; they learn a statistical understanding of language, reasoning, and eventually, visual semantics. This section explores how raw text is converted into mathematical objects and how this capability bridges the gap to computer vision.

2.10.1 Word Embeddings: From Discrete to Continuous

Neural networks cannot process strings ("cat", "dog"). They require numerical vectors. A naive approach is **One-Hot Encoding**, where "cat" is a vector of size 50,000 (vocabulary size) with a single '1'. This is inefficient and orthogonal; it implies mathematical distance between "cat" and "dog" is the same as "cat" and "car".

The Embedding Layer

LLMs utilize a **Word Embedding** layer, which is a learnable lookup matrix $E \in \mathbb{R}^{V \times d}$, where V is the vocabulary size and d is the model dimension.

$$x_{\text{embed}} = \text{Lookup}(x_{\text{token}}) \tag{2.61}$$

Through training, this layer learns to map semantically similar words to geometrically close vectors in $\mathbb{R}^d$.

$$\text{CosineSim}(E_{\text{cat}}, E_{\text{dog}}) > \text{CosineSim}(E_{\text{cat}}, E_{\text{car}}) \tag{2.62}$$

Positional Encoding in Text Generation

As established in Section 2.9.6, LLMs are permutation invariant. In text generation (e.g., GPT), position is critical.

- **Causal Masking:** In addition to Positional Encodings (Sinusoidal or RoPE), LLMs employ a **Causal Mask** in the self-attention layer. This is a lower-triangular matrix of $-\infty$ that prevents position t from attending to future positions $t+1, \ldots, T$.
- **Relative Dependency:** The positional encoding allows the model to understand syntax. For example, in "John kicked the ball," the subject "John" is identified not just by the word itself, but by its position relative to the verb "kicked" (preceding it).

2.10.2 Vision-Language Models (VLMs)

While LLMs master text and CNNs/ViTs master images, the frontier of deep learning lies in **Vision-Language Models (VLMs)**. These models learn a shared representation space where images and text are mathematically comparable.

The Modality Gap

Traditionally, an image feature vector $v \in \mathbb{R}^{512}$ (from ResNet) and a text feature vector $t \in \mathbb{R}^{512}$ (from BERT) reside in disjoint manifolds. The vector for an image of a dog and the vector for the word "dog" are unrelated. VLMs aim to align these manifolds.

2.10.3 CLIP: Contrastive Language-Image Pre-training

The seminal architecture for this alignment is **CLIP [21]**. Instead of training on fixed classes (ImageNet's 1000 labels), CLIP trains on 400 million (image, text) pairs collected from the internet.

Dual-Encoder Architecture

CLIP consists of two parallel encoders:

1. **Image Encoder (f_v):** A ResNet or ViT that maps an image I to a feature vector $v \in \mathbb{R}^d$.
2. **Text Encoder (f_t):** A Transformer that maps a text caption T to a feature vector $t \in \mathbb{R}^d$.

Both vectors are normalized to lie on a hypersphere (unit norm).

The Contrastive Loss Mechanism

CLIP does not predict labels. It learns a metric space. Given a batch of N image-text pairs $\{(I_1, T_1), \ldots, (I_N, T_N)\}$, the model predicts an $N \times N$ similarity matrix. The goal is to maximize the diagonal elements (correct pairs) and minimize the off-diagonal elements (incorrect pairings).

Let v_i and t_j be the embeddings for image i and text j. The logits are computed as scaled dot products:

$$\text{logits}_{ij} = v_i \cdot t_j \cdot e^{\tau} \tag{2.63}$$

where τ is a learnable temperature parameter.

The loss function is the symmetric Cross-Entropy loss over these similarity scores:

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}(\mathcal{L}_{\text{image}\to\text{text}} + \mathcal{L}_{\text{text}\to\text{image}}) \tag{2.64}$$

where:

$$\mathcal{L}_{\text{image}\to\text{text}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(v_i \cdot t_i \cdot e^{\tau})}{\sum_{j=1}^{N} \exp(v_i \cdot t_j \cdot e^{\tau})} \tag{2.65}$$

Why CLIP Matters: Zero-Shot Transfer

Because CLIP connects visual concepts to language, it performs **Zero-Shot Classification**. To classify an image into "cat" or "dog" without training a classifier:

1. Create prompt sentences: "A photo of a cat", "A photo of a dog".

2. Encode these sentences into vectors t_{cat}, t_{dog}.
3. Encode the image into vector v.
4. Compute similarities: $v \cdot t_{cat}$ vs $v \cdot t_{dog}$.

The text with the highest dot product is the predicted label. This allows the model to recognize categories it has never explicitly seen as labels during training, provided it understands the words describing them.

2.10.4 JEPA: Joint-Embedding Predictive Architecture

While CLIP revolutionized multimodal alignment, it relies on **Contrastive Learning**, which requires massive batches of negative samples to prevent collapse. Conversely, Generative models (like Autoencoders or MAE) rely on **Reconstruction**, predicting raw pixels. Yann LeCun proposed the **__Joint-Embedding Predictive Architecture (JEPA)__ [22]** to address the limitations of both, arguing that a model should not need to generate every pixel to understand an image.

Why Non-Decoder Architectures Work

Generative models utilize a **Decoder** to map latent representations back to pixel space ($\hat{x} \approx x$).

- **The Flaw of Reconstruction:** Predicting pixels is computationally expensive and semantically inefficient. To reconstruct an image of a dog on grass, a decoder must predict the exact position of every blade of grass. However, for semantic understanding, the model only needs to know "there is grass here," not the texture of every blade.
- **The JEPA Insight:** Instead of predicting x (pixels) from y (context), JEPA predicts the **representation** of x from the representation of y. It operates entirely in the abstract latent space, discarding the high-frequency noise that decoders are forced to model.

The Architecture: Context, Target, and Predictor

JEPA abandons the standard Encoder-Decoder framework in favor of a three-part system:

1. **Context Encoder (f_θ):** Processes the visible parts of the input x (the context) to produce a latent representation s_x.
2. **Target Encoder ($f_{\bar{\theta}}$):** Processes the masked/hidden parts of the input y to produce the target representation s_y.

3. **Predictor (g_ϕ):** A small network that takes the context representation s_x and a condition vector z (e.g., position of the mask) to predict the target representation $\hat{s}_y$.

The Learning Mechanism (EMA)

A critical challenge in joint-embedding methods without negative samples is **Model Collapse** (where the model outputs a constant vector for everything). JEPA avoids this not by negative sampling, but by using an **Exponential Moving Average (EMA)** for the Target Encoder.

The Target Encoder weights $\bar{\theta}$ are not updated via backpropagation. Instead, they slowly track the Context Encoder weights θ:

$$\bar{\theta}_t = \lambda \bar{\theta}_{t-1} + (1 - \lambda)\theta_t \tag{2.66}$$

where λ is a momentum coefficient.

The Objective: The model minimizes the distance between the *predicted* latent vector and the *actual* target latent vector:

$$\mathcal{L}_{\text{JEPA}} = \|\hat{s}_y - s_y\|_2^2 = \|g_\phi(f_\theta(x), z) - f_{\bar{\theta}}(y)\|_2^2 \tag{2.67}$$

By forcing the student (f_θ, g_ϕ) to predict the representation of the teacher $(f_{\bar{\theta}})$, the model learns to encode high-level semantic structures (shapes, objects) while ignoring unpredictable low-level details (noise, texture) that are filtered out by the encoding process.

Chapter 3

Generative models

3.1 Generative Adversarial Network (GAN)

3.1.1 Motivation: Beyond Pattern Matching

In previous sections, we explored discriminative models that map inputs to labels ($x \to y$) or reconstruction-based models like Autoencoders (and JEPA, implicitly) that aim to recover signal from noise. However, a true "understanding" of a dataset implies the ability to create it. The goal of Generative Modeling is to learn the underlying data distribution P_{data} and sample new, realistic instances from it.

While Variational Autoencoders (VAEs) maximize a lower bound on data likelihood, **Generative Adversarial Networks (GANs)** proposed a radical shift. Instead of minimizing a reconstruction error (which often leads to blurry outputs), GANs train a model using a competitive game.

3.1.2 The Architecture: The Counterfeit Game

A GAN consists of two neural networks trained simultaneously in a zero-sum game framework:

- **The Generator (G):** Often likened to a "Counterfeiter." It takes a random noise vector z (sampled from a prior distribution p_z, typically Gaussian or Uniform) and maps it to the data space to produce a fake sample:

  $$x_{fake} = G(z)$$

  Its goal is to generate samples indistinguishable from real data to "fool" the discriminator.

- **The Discriminator (D):** Likened to the "Police." It takes an input x and outputs a probability $D(x) \in [0, 1]$.

  $$D(x) = P(Input \text{ is Real}|x)$$

Its goal is to correctly classify real samples (drawn from P_{data}) as 1 and fake samples (drawn from G) as 0.

3.1.3 The Minimax Loss Function

The training process is formulated as a minimax game where D tries to maximize the objective value function $V(D, G)$, and G tries to minimize it. The value function is defined as:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))] \tag{3.1}$$

Mathematical Breakdown:

1. **Discriminator Perspective ($\max_D$):** D wants to maximize $\log D(x)$ (pushing the prediction for real data towards 1 (real)) and maximize $\log(1 - D(G(z)))$ (pushing the prediction for fake data towards 0 (fake)).

2. **Generator Perspective ($\min_G$):** The first term $\mathbb{E}_{x \sim p_{data}}[\dots]$ depends only on real data, so G cannot influence it. G focuses entirely on minimizing the second term $\log(1 - D(G(z)))$. By minimizing the likelihood that D is correct about fakes, G forces D to output high probabilities for generated data.

3.1.4 Limitation: Mode Collapse

While GANs are capable of generating sharp images, they suffer from a notorious instability known as **Mode Collapse**. This occurs when the Generator finds a specific output (or a small set of outputs) that successfully fools the Discriminator and begins to over-produce it, ignoring the diversity of the true data distribution.

The Mechanism of Collapse: Instead of learning the entire distribution P_{data}, the Generator maps the entire latent space z to a single high-probability region in x.

- The Generator produces a single realistic image (Mode A) that fools D.
- The Discriminator learns to identify Mode A specifically as fake.
- The Generator simply switches to a different image (Mode B) to fool D again, abandoning Mode A.

This results in a "cat-and-mouse" cycle where the generator hops between modes but never captures the full richness of the dataset (e.g., generating only "shoes" in a dataset of clothing), limiting its utility in tasks requiring diverse creativity.

3.2 Autoencoders (AE)

3.2.1 Motivation: Learning Efficient Representations

The primary motivation for Autoencoders is **Dimensionality Reduction** and **Feature Learning**. Unlike Principal Component Analysis (PCA), which is limited to linear transformations, Autoencoders can learn non-linear manifolds, allowing them to compress complex data (like images) into a compact "bottleneck" representation that captures the most essential variations of the data.

3.2.2 The Architecture and Mathematics

An Autoencoder consists of two distinct components joined by a bottleneck:

1. **The Encoder (f_ϕ):** Compresses the high-dimensional input $x \in \mathbb{R}^D$ into a lower-dimensional latent code $z \in \mathbb{R}^d$ (where $d \ll D$).
2. **The Decoder (g_θ):** Attempts to reconstruct the original input from the latent code, producing $\hat{x}$.

Mathematical Formulation: Let the encoder be parameterized by ϕ and the decoder by θ. The forward pass is defined as:

$$z = f_\phi(x) = \sigma(W_{enc}x + b_{enc}) \tag{3.2}$$

$$\hat{x} = g_\theta(z) = \sigma'(W_{dec}z + b_{dec}) \tag{3.3}$$

Here, σ represents non-linear activation functions (like ReLU or Sigmoid).

The Objective Function: The network is trained to minimize the **Reconstruction Loss** $\mathcal{L}$, which measures the difference between the original input x and the reconstruction $\hat{x}$. For continuous data, the Mean Squared Error (MSE) is typically used:

$$\mathcal{L}_{AE}(\theta, \phi) = \frac{1}{N}\sum_{i=1}^{N} ||x^{(i)} - g_\theta(f_\phi(x^{(i)}))||_2^2 \tag{3.4}$$

By forcing the information through a constrained bottleneck ($d < D$), the network is forced to prioritize the most significant features (signal) and ignore the noise.

3.2.3 Limitations: The Generative Gap

While Autoencoders are excellent for compression and denoising, they fail as true **Generative Models** due to two primary limitations:

1. **Discontinuous Latent Space:** Standard AEs do not enforce any structure on the latent space Z. The training points are mapped to discrete clusters. If you sample a random point z from the "empty space" between clusters and decode it, the output is often garbage noise. This makes interpolation impossible.
2. **The "Blurry" Output Problem:** The use of MSE loss encourages the model to find the "average" pixel value to minimize error.

   If the model is unsure whether a pixel should be sharp black or white (e.g., an edge), the mathematically safe bet is gray. This results in reconstructions that lack high-frequency details and appear blurry compared to GANs.

To solve the latent space discontinuity, we typically upgrade to **Variational Autoencoders (VAEs)**, which force z to follow a Gaussian distribution.

3.3 Variational Autoencoders (VAE)

3.3.1 Motivation: From Compression to Generation

As discussed in the previous section, standard Autoencoders learn a discontinuous latent space where random sampling yields "garbage" outputs. **Variational Autoencoders (VAEs)** solve this by imposing a probabilistic structure on the latent space.

Instead of mapping an input x to a fixed point z, a VAE maps it to a **probability distribution** (typically a Gaussian). This forces the latent space to be continuous and smooth, allowing us to sample new, coherent data points from the "gaps" between training examples, effectively turning the Autoencoder into a Generative Model.

3.3.2 The Architecture and Mathematics

The VAE consists of a probabilistic encoder and decoder:

- **Probabilistic Encoder $q_\phi(z|x)$:** Instead of outputting a code z, the encoder outputs two vectors: a mean μ and a variance σ^2. These parameterize the approximate posterior distribution.
- **Reparameterization Trick:** To allow backpropagation through a random sampling process, we express z as a deterministic transformation of a noise variable $\epsilon \sim \mathcal{N}(0, I)$. Formally, the encoder will condense input x to μ and σ, and $\sigma(\cdot)$ denote activation function of choice (ReLU, GELU, SiLU, Tanh). We have a distribution of encoded latent code z in a form of a distribution:

  $$z = \mu + \sigma \odot \epsilon \tag{3.5}$$

  where:

  $$\mu = f_{\phi_\mu}(x) = \sigma(W_\mu x + b_\mu) \tag{3.6}$$
  $$\sigma = f_{\phi_\epsilon}(x) = \sigma(W_\sigma x + b_\sigma) \tag{3.7}$$

- **Probabilistic Decoder $p_\theta(x|z)$:** Reconstructs the input from the sampled z.

The Evidence Lower Bound (ELBO): We cannot directly maximize the likelihood of the data. Instead, we maximize the Evidence Lower Bound (ELBO), which serves as the loss function (minimized):

$$\mathcal{L}_{VAE} = \underbrace{-\mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)]}_{\text{Reconstruction Loss}} + \underbrace{D_{KL}(q_\phi(z|x)||p(z))}_{\text{Regularization Term}} \tag{3.8}$$

- **Reconstruction Loss:** Encourages the decoder to recover the data (similar to AE).
- **KL Divergence:** Forces the learned distribution $q_\phi(z|x)$ to be close to the prior unit Gaussian $p(z) = \mathcal{N}(0, I)$. This "packs" the clusters close together, eliminating dead zones in the latent space.

3.3.3 Limitations: The Blurry Trade-off

While VAEs solve the sampling problem of standard AEs and are more stable to train than GANs, they suffer from distinct limitations:

1. **Blurry Generations:** Like standard AEs, VAEs typically use a likelihood objective (like MSE). This averages out details, leading to images that look "soft" or out of focus compared to the sharp, hallucinated details of a GAN.
2. **Posterior Collapse:** If the decoder is too powerful (e.g., an autoregressive model), it may learn to ignore the latent code z entirely. The KL term drives the posterior to exactly match the prior, carrying zero information about the input.

3.4 Denoising Diffusion Probabilistic Models (DDPM)

3.4.1 Motivation: The Generative Trilemma

In the previous chapters, we explored the trade-offs of generative modeling. We faced a "Trilemma" where existing models could typically achieve only two of the following three goals:

1. **High Sample Quality:** Sharp, realistic images (GANs).
2. **Mode Coverage:** Capturing the full diversity of the dataset (VAEs).
3. **Training Stability:** Converging without adversarial instability (VAEs).

GANs offer high quality and speed but suffer from mode collapse and training instability. **VAEs** offer excellent diversity and stability but suffer from blurry outputs due to the MSE/likelihood objective.

Diffusion Models, inspired by non-equilibrium thermodynamics, emerged to solve this. They offer both high sample quality (competing with GANs) and high mode coverage (like VAEs), with the trade-off of being computationally expensive at inference time.

3.4.2 The Intuition: Reversing Entropy

Physically, diffusion describes a process where structure turns into chaos—like a drop of ink dispersing into a glass of water until it becomes uniform.

Diffusion models ask the reverse question: *Can we learn to reverse time? Can we look at a glass of cloudy water and mathematically reconstruct the original ink drop?*

3.4.3 Mathematical Formulation

DDPMs define two Markov chains: a fixed forward process that destroys data, and a learned reverse process that creates data.

The Forward Process (Diffusion)

Given a data point $x_0 \sim q(x)$, we gradually add Gaussian noise over T steps according to a fixed variance schedule β_t. The distribution of x_t given x_{t-1} is:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t\mathbf{I}) \tag{3.9}$$

Crucially, we can sample x_t at any arbitrary timestep t directly from x_0 without iterating through intermediate steps:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I}) \tag{3.10}$$

Where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ and $\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$.

As $t \to \infty$, the signal is destroyed, and x_T approaches pure noise $\mathcal{N}(0, \mathbf{I})$.

The Reverse Process (Denoising)

To generate data, we sample pure noise x_T and learn to remove the noise step-by-step. Since the exact posterior $q(x_{t-1}|x_t)$ is intractable, we approximate it with a parameterized model p_θ (typically a U-Net):

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \tag{3.11}$$

3.4.4 The Loss Function

While the mathematical derivation involves maximizing the Evidence Lower Bound (similar to VAEs), Ho et al. demonstrated a powerful simplification.

Instead of predicting the image mean μ_θ directly, the network is trained to predict the **noise** ϵ that was added to the image. The loss function becomes a simple weighted Mean Squared Error (MSE):

$$\mathcal{L}_{simple}(\theta) = \mathbb{E}_{x_0,\epsilon\sim\mathcal{N}(0,1),t}\left[||\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)||^2\right] \tag{3.12}$$

Interpretation:

- We take a real image x_0 and corrupt it with random noise ϵ to create x_t.
- We give x_t to the U-Net and ask: "What noise did I add?"
- The network outputs ϵ_θ, and we penalize the difference from the actual noise ϵ.

This transforms the complex generative task into a supervised regression problem, ensuring high training stability.

3.4.5 Image Construction: The Generation Loop

Now that we have derived the mathematical update rule, we can define the full algorithm for constructing an image from thin air.

The process is an **iterative refinement loop**. Unlike GANs, which generate an image in a single forward pass, Diffusion Models typically require T passes (e.g., 1000 steps) to slowly carve the image out of the noise.

The Algorithm (Sampling)

The generation process follows these specific steps:

Algorithm 1 Sampling (Generation)

Input: A trained noise predictor network $\epsilon_\theta(x_t, t)$
Start: Sample pure Gaussian noise $x_T \sim \mathcal{N}(0, \mathbf{I})$
for $t = T, T-1, \ldots, 1$ **do**
    Sample $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ (if $t > 1$, else $\mathbf{z} = 0$)
    Predict Noise: $\epsilon_{pred} = \epsilon_\theta(x_t, t)$
    Denoise: Compute x_{t-1} using the update rule:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{pred} \right) + \sigma_t \mathbf{z}$$

end for
Return x_0 (The generated image)

Visualizing the Evolution

The construction of the image follows a "Coarse-to-Fine" trajectory. Because the noise schedule β_t is defined mathematically, different frequency details emerge at specific stages of the loop:

- **Phase 1: Structure Formation** ($t \approx 1000 \to 800$)
  The input is pure static. The model makes large, aggressive updates to the global structure. At this stage, you might see vague blobs of color or the outline of a composition (e.g., "sky goes here, ground goes here").
- **Phase 2: Content Definition** ($t \approx 800 \to 400$)
  The semantic content becomes recognizable. The blobs turn into distinct objects. A face will have eyes and a mouth, but they will look grainy and low-resolution. The variance (noise) is still high enough to allow the model to change its mind about minor details.
- **Phase 3: Detail Refinement** ($t \approx 400 \to 0$)
  The global structure is now locked in. The model focuses on high-frequency details: texture of hair, reflections in eyes, or the grain of wood. The noise added ($\sigma_t \mathbf{z}$) is very small, serving only to polish the final pixels.

Key Takeaway: This slow, iterative process is why Diffusion Models have such high mode coverage. By not committing to a final image immediately (like a GAN), the model can explore the probability space more thoroughly before "collapsing" onto a specific, high-quality result.

3.5 The Generative Trilemma

Just as the "CAP Theorem" dictates trade-offs in distributed systems, Generative AI faces its own fundamental constraint: the **Generative Trilemma**.

Research suggests that it is mathematically difficult for a single model to simultaneously achieve all three of the following goals:

1. **High Sample Quality:** Realistic, sharp, and detailed images (High Fidelity).
2. **Fast Sampling Speed:** Generating data in real-time or with few computational steps.
3. **Mode Coverage (Diversity):** Capturing the entire variety of the data distribution (no dropped modes).

3.5.1 The Trade-offs

Each of the three major families we have studied occupies a different "corner" of this triangle, optimizing for two traits at the expense of the third.

- **GANs (Generative Adversarial Networks):**
  - **Strengths:** *Quality + Speed.* GANs produce the sharpest images and do so in a single forward pass (milliseconds).
  - **Weakness:** *Diversity.* They suffer from Mode Collapse, often ignoring difficult parts of the data distribution to focus on what is easiest to fake.
- **VAEs (Variational Autoencoders):**
  - **Strengths:** *Speed + Diversity.* VAEs are fast (single pass) and mathematically guarantee good mode coverage due to the likelihood-based loss.
  - **Weakness:** *Quality.* The element-wise MSE loss and the Gaussian assumption often result in blurry or "fuzzy" outputs compared to GANs.
- **Diffusion Models (DDPM/Score-Based):**
  - **Strengths:** *Quality + Diversity.* Diffusion models have broken the curse of Mode Collapse while matching (or exceeding) the sharpness of GANs.
  - **Weakness:** *Speed.* The iterative refinement process requires running the full network hundreds or thousands of times to generate a single image, making them orders of magnitude slower than GANs or VAEs.

3.5.2 Current Research Direction

Modern research is focused on breaking this trilemma. Techniques like **Latent Diffusion Models (LDM)** and **Consistency Models** aim to maintain the Quality and Diversity of diffusion while drastically reducing the sampling steps (improving Speed), effectively trying to merge the strengths of all three families.

3.6 The Mathematics of Generative Loss Functions

The core objective of generative modeling is to learn the true data distribution $p_{data}(x)$. However, the method of approximation differs fundamentally across architectures. This section derives the loss functions for AE, VAE, GAN, and Diffusion models, demonstrating how each is constructed to solve specific intractability problems.

3.6.1 Autoencoders: Deterministic Maximum Likelihood

Construction: The standard Autoencoder minimizes the Mean Squared Error (MSE) between the input x and the reconstruction $\hat{x}$. This is not arbitrary; it is derived from the principle of Maximum Likelihood Estimation (MLE) under the assumption of a fixed variance Gaussian noise model.

Assume the decoder $p_\theta(x|z)$ outputs a Gaussian distribution centered at $D(z)$ with fixed identity covariance $\sigma^2\mathbf{I}$:

$$p(x|z) = \frac{1}{(2\pi\sigma^2)^{d/2}} \exp\left(-\frac{||x - D(z)||^2}{2\sigma^2}\right)$$

Minimizing the negative log-likelihood (NLL):

$$\mathcal{L}_{AE} = -\log p(x|z) = \underbrace{\frac{d}{2}\log(2\pi\sigma^2)}_{\text{Constant}} + \frac{1}{2\sigma^2}||x - D(E(x))||^2$$

Dropping constants, we recover the standard reconstruction loss:

$$\mathcal{L}_{AE} \propto ||x - \hat{x}||^2 \tag{3.13}$$

Computational Efficiency:

- **Training/Inference:** Highly efficient ($\mathcal{O}(1)$). Requires a single forward pass.
- **Drawback:** The assumption of a Gaussian output distribution is unimodal. It forces the model to average conflicting modes, resulting in blurriness.

3.6.2 VAEs: The Variational Lower Bound (ELBO)

Construction: The goal of the Variational Autoencoder is to maximize the marginal likelihood of the data, $p_\theta(x)$. However, the integral required to compute this is intractable:

$$p_\theta(x) = \int p_\theta(x|z)p(z)dz$$

To construct a tractable loss function, we employ two specific mathematical techniques: **Importance Sampling** and **Jensen's Inequality**.

Importance Sampling with $q_\phi(z|x)$

Since integrating over the entire latent space $p(z)$ is inefficient (most z do not generate valid x), we introduce an approximate posterior $q_\phi(z|x)$ (the encoder) to focus on relevant regions. We multiply and divide the integrand by this term:

$$\log p_\theta(x) = \log \int p_\theta(x,z) \frac{q_\phi(z|x)}{q_\phi(z|x)} dz = \log \mathbb{E}_{z\sim q_\phi}\left[\frac{p_\theta(x,z)}{q_\phi(z|x)}\right] \tag{3.14}$$

Jensen's Inequality

The logarithm of an expectation, $\log \mathbb{E}[\cdot]$, is analytically difficult. We apply **Jensen's Inequality** for concave functions ($f(\mathbb{E}[y]) \geq \mathbb{E}[f(y)]$) to move the logarithm *inside* the expectation. This provides a valid Lower Bound:

$$\log p_\theta(x) \geq \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x,z)}{q_\phi(z|x)}\right] \tag{3.15}$$

The ELBO Objective

Expanding the logarithm term $\log(ab/c) = \log a + \log b - \log c$, we derive the Evidence Lower Bound (ELBO):

$$\begin{aligned} \text{ELBO} &= \mathbb{E}_q[\log p_\theta(x|z)] + \mathbb{E}_q[\log p(z) - \log q_\phi(z|x)] && (3.16)\\ &= \underbrace{\mathbb{E}_q[\log p_\theta(x|z)]}_{\text{Reconstruction Term}} - \underbrace{D_{KL}(q_\phi(z|x)||p(z))}_{\text{Regularization Term}} && (3.17) \end{aligned}$$

We maximize this lower bound (or minimize its negative) as a proxy for the true likelihood.

Computational Efficiency:

- **Training:** Highly efficient ($\mathcal{O}(1)$). The expectations are approximated via Monte Carlo sampling using the **Reparameterization Trick** ($z = \mu + \sigma \odot \epsilon$), which allows gradients to backpropagate through the stochastic sampling node.
- **Inference:** Fast ($\mathcal{O}(1)$). Generating data requires only sampling $z \sim p(z)$ and running the decoder once.

3.6.3 Deconstructing the GAN Loss: The Dual Objective

Unlike Autoencoders or Diffusion models which optimize a single objective function, GANs optimize two separate, competing loss functions simultaneously.

The Discriminator Loss ($\mathcal{L}_D$)

Goal: The Discriminator acts as a binary classifier. Its sole objective is to correctly categorize input data into two classes: Real (1) and Fake (0).

Construction: We use standard **Binary Cross Entropy (BCE)**. The loss is the sum of two errors:

- Error on Real Data: It wants $D(x) \to 1$.
- Error on Fake Data: It wants $D(G(z)) \to 0$.

$$\mathcal{L}_D = \underbrace{-\mathbb{E}_{x\sim p_{data}}[\log D(x)]}_{\text{Maximize prob. of Real}} \underbrace{-\mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]}_{\text{Maximize prob. of Fake}} \tag{3.18}$$

Computational Efficiency:

- **Cost:** $\mathcal{O}(1)$ per batch.
- **Operation:** Requires one forward pass of real data through D, one forward pass of latent noise through G then D, and backpropagation only through D.

The Generator Loss ($\mathcal{L}_G$)

Goal: The Generator wants to fool the Discriminator. It wants the fake images $G(z)$ to be classified as Real (1).

Construction (The "Non-Saturating" Heuristic): In the original Minimax game, the Generator tries to minimize the Discriminator's success: $\min[\log(1-D(G(z)))]$. However, early in training, when D is very smart and G is very dumb, $D(G(z)) \approx 0$. The gradient of this function vanishes (slope becomes flat), and G stops learning.

To fix this, we use the **Non-Saturating Loss**. Instead of minimizing the likelihood of being fake, we maximize the likelihood of being real:

$$\mathcal{L}_G = -\mathbb{E}_{z\sim p_z}[\log D(G(z))] \tag{3.19}$$

This is to ensure we give the reward to the Generator G if it fool the Discriminator.

Computational Efficiency:

- **Cost:** $\mathcal{O}(1)$ per batch.
- **Operation:** Requires forwarding noise through G, then D. Crucially, gradients must flow *through* D back to G, but we do not update D's weights during this step.

3.6.4 Diffusion Models: Reweighted Variational Bound

Construction: Diffusion models are mathematically VAEs with a fixed encoder and a hierarchical latent structure $z = \{x_1, \ldots, x_T\}$. The exact variational bound L_{vlb} consists of T KL-divergence terms:

$$L_{vlb} = \sum_{t=1}^{T} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))$$

Ho et al. simplified this by parameterizing the model to predict the noise ϵ rather than the mean μ. The term-wise loss for step t becomes:

$$L_t \propto ||\epsilon - \epsilon_\theta(x_t, t)||^2$$

Crucially, the authors found that discarding the weighting coefficients required by the strict variational bound improves sample quality. This leads to the "Simple Loss":

$$\mathcal{L}_{DM} = \mathbb{E}_{t,x_0,\epsilon}\left[||\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)||^2\right] \tag{3.20}$$

Computational Efficiency:

- **Training:** Efficient. It is reduced to a stable regression problem (MSE) with no adversarial instability.
- **Inference: Highly Inefficient** ($\mathcal{O}(T)$). Generating a single sample requires iterating the network T times (e.g., 1000 steps), making it orders of magnitude slower than GANs or VAEs.

Chapter 4

eXplainable AI—we need to know AI decision reasons

4.1 Interpretability: SHAP and Game Theory

As Deep Learning models grow in complexity (from simple MLPs to billion-parameter Transformers), they become "Black Boxes." While they provide accurate predictions, they fail to explain *why* a specific prediction was made. To trust these models in critical domains (healthcare, finance), we require rigorous interpretability frameworks. The state-of-the-art method is **SHAP (SHapley Additive exPlanations)**[23].

4.1.1 Theoretical Foundation: Cooperative Game Theory

SHAP is not merely a heuristic; it is founded on **Cooperative Game Theory**, specifically the work of Lloyd Shapley (Nobel Prize 2012).

The Shapley Value

Consider a game where a coalition of players cooperates to generate a total payout (gain). The fundamental question Shapley answered is: **"How do we fairly distribute the total payout among the players based on their contribution?"**

Let F be the set of all players. Let $v(S)$ be a **Characteristic Function** that maps a subset of players (a coalition) $S \subseteq F$ to a real-valued payout. The **Shapley Value** ϕ_i for player i is the average marginal contribution of player i across all possible coalitions.

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[v(S \cup \{i\}) - v(S) \right] \tag{4.1}$$

Mathematical Breakdown:

1. **Marginal Contribution:** The term $[v(S \cup \{i\}) - v(S)]$ represents how much the payout increases when player i joins the coalition S.
2. **Combinatorial Weight:** The fraction $\frac{|S|!(|F|-|S|-1)!}{|F|!}$ is the probability that coalition S is formed randomly before player i joins. It ensures that every permutation of players is weighted equally.

Axiomatic Properties

Shapley proved that this formula is the *only* distribution method that satisfies four axioms of fairness:

- **Efficiency:** $\sum \phi_i = v(F)$. The sum of individual attributions equals the total payout.
- **Symmetry:** If two players contribute equally to all coalitions, their values are equal.
- **Dummy:** If a player contributes zero to every coalition, their value is zero.
- **Additivity:** The value for a sum of games is the sum of the values.

4.1.2 Applying SHAP to Machine Learning

Scott Lundberg (2017) adapted this framework to Machine Learning by mapping the components:

- **Game:** The prediction task for a single data point x.
- **Players:** The input features (pixels, words, tabular columns).
- **Payout $v(S)$:** The expected model prediction given only the features in subset S are known.

The Value Function in ML

In ML, we cannot simply "remove" a feature (a neural network requires fixed input dimensions). Instead, we simulate "missing" features by marginalizing over a background dataset. For a model f and a subset of features S, the characteristic function is the conditional expectation:

$$v(S) = \mathbb{E}[f(x) \mid x_S] \tag{4.2}$$

The SHAP value ϕ_i tells us how much feature i shifts the prediction away from the global average prediction (the base rate).

$$f(x) = \phi_0 + \sum_{i=1}^{M} \phi_i \tag{4.3}$$

Where $\phi_0 = \mathbb{E}[f(x)]$ is the average prediction of the dataset.

4.1.3 DeepSHAP: Efficient Approximation for DL

Calculating the exact Shapley value is NP-Hard (2^M possible coalitions). For deep neural networks with thousands of inputs, this is computationally impossible.

DeepSHAP combines Shapley values with **DeepLIFT** (Deep Learning Important FeaTures), a backpropagation-based interpretation method. DeepSHAP approximates the conditional expectation by assuming feature independence and linearizing the non-linear components (ReLU, Sigmoid) of the network.

How to Apply DeepSHAP

In practice (e.g., using the `shap` library in Python), the process involves:

1. **Background Selection:** Select a summary of the training data (e.g., 100 random samples or K-means centroids). This serves as the "reference state" (what the model expects to see on average).
2. **Forward-Backward Pass:** For a specific input x, DeepSHAP performs a forward pass to compute activations and a backward pass to distribute the "difference from reference" down to the input features.
3. **Interpretation:**
   - **Local:** Waterfall plots show why *this specific* image was classified as a "Cat" (e.g., $+0.4$ from the ears, $+0.2$ from the whiskers, -0.1 from the background color).
   - **Global:** Summary plots aggregate SHAP values across the test set to show global feature importance (e.g., "High blood pressure generally increases stroke risk").

Why SHAP is Crucial: Addressing Non-Linearity and Interactions

It is critical to understand why simpler interpretation methods—such as analyzing weight matrices or computing *Saliency Maps* (gradients $\nabla_x f(x)$)—are mathematically insufficient for Deep Learning. SHAP addresses three fundamental pathologies inherent in these naive approaches.

1. The Failure of Gradient-Based Sensitivity (Saturation) In Deep Neural Networks, activation functions like Sigmoid or Tanh saturate. Consider a simple neuron $y = \sigma(w \cdot x + b)$. If the input x is very large, the output saturates at $y \approx 1$. At this saturation point, the local gradient is effectively zero:

$$\frac{\partial y}{\partial x} \approx 0 \tag{4.4}$$

A standard Saliency Map (which plots gradients) would erroneously report that feature x has **zero importance**, despite x being the sole cause of the neuron firing.

- **The SHAP Solution:** SHAP does not measure local sensitivity. It measures the **marginal contribution** relative to a baseline. By asking "What is the output with x versus the output without x?", SHAP correctly identifies that x drives the output from 0.5 (baseline) to 1.0, assigning it high importance regardless of the vanishing gradient.

2. Capturing Interaction Effects (The XOR Problem) Deep Learning excels because it models complex interactions between features. Simple interpretation methods often assume *additivity*, treating features as independent contributors. Consider a mathematical function modeling an Exclusive OR (XOR) logic, which is common in image recognition (e.g., "A pixel is only an edge if its neighbor is dark"):

$$f(x_1, x_2) = x_1 \oplus x_2 \tag{4.5}$$

Suppose inputs are binary $\{0, 1\}$.

- **Scenario:** Input is $(1, 1)$, so Output is 0.
- **Naive Perturbation:** If we toggle x_1 to 0, the input becomes $(0, 1)$, and the output becomes 1. The change is $+1$. If we toggle x_2, the change is also $+1$. A naive method might confuse the direction of influence.
- **Shapley Coalition:** SHAP averages the contribution of x_1 over *all possible subsets* of other features (x_2 present vs. x_2 absent).

  $$\phi_{x_1} \propto [f(1, 1) - f(0, 1)] + [f(1, 0) - f(0, 0)] \tag{4.6}$$

  This combinatorial sampling captures the fact that x_1's effect **flips sign** depending on the state of x_2. This ability to model non-additive interactions is what makes SHAP the only method capable of faithfully explaining architectures like Transformers, where Self-Attention is inherently interaction-based.

3. Mathematical Consistency Perhaps most importantly for CS theory, SHAP is the unique solution satisfying the property of **Consistency**. Imagine two models, Model A and Model B. Suppose that for every possible input combination, changing feature x_i increases the output of Model A *more* than it increases the output of Model B.

$$\forall z, \quad f_A(z') - f_A(z) \geq f_B(z') - f_B(z) \tag{4.7}$$

Intuitively, the importance score for feature x_i should be higher for Model A. **Gradient methods violate this axiom.** SHAP is mathematically proven to be the *only* attribution method that guarantees this consistency, ensuring that if we change the model architecture to rely more on a specific feature, the interpretation reflects that change.

4.1.4 Limitations of SHAP: The Feature Space Disconnect

While SHAP provides a mathematically unique solution for attributing credit to *input features*, it faces a fundamental limitation when applied to Deep Learning: the disconnect between the **Input Space** (pixels, tokens) and the **Latent Space** (concepts, semantics).

The Semantic Gap

Let $f : \mathcal{X} \to \mathcal{Y}$ be a deep neural network, decomposable into a feature extractor $g : \mathcal{X} \to \mathcal{Z}$ and a classifier $h : \mathcal{Z} \to \mathcal{Y}$, such that $f(x) = h(g(x))$.

- $\mathcal{X}$: High-dimensional, low-level input (e.g., 224×224 pixels).
- $\mathcal{Z}$: Lower-dimensional, high-level semantic manifold (e.g., 512-d vector representing "furry", "ears", "indoors").

SHAP computes attribution values $\phi \in \mathcal{X}$. It tells us *which pixels* matter. However, humans reason in concepts ($\mathcal{Z}$), not pixels.

$$\text{SHAP}(f, x) \to \text{Heatmap of Pixels} \tag{4.8}$$

The Problem: A SHAP heatmap might highlight a patch of green pixels to explain a "Dog" prediction. Does the model see "Grass" (contextual correlation) or "Tennis Ball" (object)? SHAP cannot distinguish between these latent concepts because it collapses the rich, non-linear processing of $g(x)$ back onto the flat input space. It explains *where* the model looked, but not *what* concept it detected.

Correlation vs. Causation in Latent Space

Deep networks often learn spurious correlations in the latent space (e.g., "Boats are always on water"). If feature x_i (water pixels) is highly correlated with feature x_j (boat pixels) in the training distribution P_{data}, the Shapley value calculation—which assumes it can toggle features independently—may produce misleading results.

1. **Out-of-Distribution (OOD) Sampling:** To compute the marginal contribution of a pixel, SHAP must simulate "removing" it. In practice, this means replacing it with a baseline value (e.g., grey).
2. **The Manifold Violation:** The resulting sample $x'_{distorted}$ may lie off the data manifold $\mathcal{Z}$. The model $f(x'_{distorted})$ produces an arbitrary output (extrapolation error).

Consequently, SHAP may assign high positive importance to background pixels simply because replacing them breaks the learned latent structure of the image, causing the probability to drop. The explanation reflects the **fragility of the model** rather than the **true semantic cause** of the prediction. To address this, research is shifting toward *Concept Activation Vectors (TCAV)* which operate directly in $\mathcal{Z}$.

4.2 Local Interpretability: LIME

While SHAP offers a theoretically unified solution based on Game Theory, it can be computationally expensive. An alternative, and often more intuitive approach, is **LIME

(Local Interpretable Model-agnostic Explanations)**[24]. Proposed by Ribeiro et al. (2016), LIME operates on a fundamental insight: **complex, non-linear models behave linearly if you zoom in close enough.**

4.2.1 The Philosophy: Local Fidelity vs. Global Consistency

A Deep Neural Network (DNN) describes a highly non-convex, non-linear decision boundary across the entire input space. Explaining the *entire* model with a simple rule is impossible (if it were possible, we wouldn't need a DNN).

However, for a *single* prediction x, we do not care about the global boundary. We only care about the boundary's shape in the immediate vicinity of x. LIME aims to train a **Local Surrogate Model**—an interpretable model (like Linear Regression or a Decision Tree)—that mimics the behavior of the Black Box model f, but *only* in the neighborhood of x.

4.2.2 The Mathematical Formulation

Let f be the complex model being explained. Let $g \in G$ be an interpretable model class (e.g., linear models). Let $\pi_x(z)$ be a proximity measure (kernel) that defines how close a sample z is to the original instance x.

LIME seeks the model g that minimizes the following objective:

$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \tag{4.9}$$

Components:

- $\mathcal{L}(f, g, \pi_x)$: The **Fidelity Loss**. It measures how inaccurate g is at approximating f in the locality defined by π_x. Typically, this is weighted squared loss.
- $\Omega(g)$: The **Complexity Penalty**. We want the explanation to be simple. For a linear model, this might be the number of non-zero coefficients (sparsity).

4.2.3 The Algorithm: How LIME Works

Since LIME is model-agnostic, it cannot peek inside the neural network (no gradients). It treats f purely as a function $f : X \to Y$.

1. **Perturbation (Sampling):** Given the instance x (e.g., an image), LIME generates a dataset of N perturbed samples $\{z_1, \ldots, z_N\}$.
    - For images, this involves randomly graying out "super-pixels" (contiguous patches).
    - For text, this involves randomly removing words.

2. **Black Box Prediction:** We feed these perturbed samples into the deep network to get predictions: $y'_i = f(z_i)$.

3. **Weighting:** We calculate weights $w_i = \pi_x(z_i)$ based on similarity. Samples that look like the original x get high weights; samples that are very different get low weights.

4. **Fitting the Surrogate:** We train a weighted linear regression model (usually LASSO to enforce sparsity) on the dataset $\{(z_i, y'_i)\}$.

$$g(z) = w \cdot z + b \tag{4.10}$$

5. **Explanation:** The coefficients w of this simple linear model serve as the explanation. If the coefficient for the "nose" super-pixel is highly positive, it means the nose locally pushes the prediction toward the target class.

4.2.4 Utility in ML and Deep Learning

Why is this local approximation useful?

Debugging and Trust (The "Husky" Problem)

The classic example of LIME's utility involved a classifier distinguishing Wolves from Huskies. The model had high accuracy. However, LIME explanations revealed that for "Wolf" predictions, the model was focusing **entirely on the snow** in the background, ignoring the animal itself. Without local interpretation, we would assume the model learned "Wolf features." LIME exposed that it actually learned "Snow detector," preventing the deployment of a flawed model.

Regulatory Compliance (Right to Explanation)

In sectors like banking or healthcare, regulations (e.g., GDPR) often grant users a "right to explanation." You cannot deny a loan simply because "the AI said so." LIME provides a legally defensible local rationale: *"The loan was denied primarily because the Debt-to-Income ratio (Feature A) was too high, despite the High Savings (Feature B)."*

Stability Analysis

By running LIME on similar inputs, we can assess the stability of the model. If a small amount of noise changes the LIME explanation drastically (e.g., the model suddenly switches from focusing on "eyes" to focusing on "ears"), it indicates the decision boundary is jagged and the model is likely overfitting or fragile.

4.3 How XAI is Used in Industry

In the academic setting, XAI is often treated as a diagnostic tool. In the industrial setting, however, it is a critical component of the deployment pipeline, serving distinct roles in trust building, privacy auditing, and iterative development.

4.3.1 Explainability: High-Stakes Decision Making

The primary industrial use case for XAI is bridging the "Trust Gap" between AI systems and human experts in high-stakes domains.

Healthcare: Clinical Decision Support

In medical imaging (e.g., detecting diabetic retinopathy or tumors), a "black box" prediction of "99% Malignant" is insufficient for a doctor to authorize surgery.

- **Application:** Radiologists use **Saliency Maps** or **Grad-CAM** to overlay the model's attention on the MRI scan.
- **Example:** If a model predicts lung cancer, the XAI layer highlights the specific nodule it detected. If the model highlights a rib bone or a scanner artifact instead of tissue, the doctor knows to reject the diagnosis. This "Human-in-the-loop" workflow requires interpretability to function safely.

Finance: Adverse Action Notices

When a bank denies a loan application using an algorithmic credit scoring model, they are legally required (in jurisdictions like the US and EU) to provide an "Adverse Action Notice" explaining *why*.

- **Application:** Banks run SHAP on the denied applicant's profile.
- **Example:** Instead of a generic rejection, the system generates a personalized explanation: *"Your loan was denied primarily because your Debt-to-Income ratio (0.45) exceeds the threshold, and your Credit History Length (2 years) is below the median."* This transparency turns a binary rejection into actionable feedback for the customer.

4.3.2 Privacy and Fairness Auditing

While XAI is often associated with transparency, it plays a nuanced role in **Data Privacy** and **Algorithmic Fairness**.

Detecting Proxy Variables (Fairness)

Laws often prohibit models from using protected attributes like Race or Gender. However, deep learning models are adept at finding "proxies" (e.g., using Zip Code as a proxy for Race).

- **Application:** XAI is used to audit the model before deployment. By analyzing global feature importance (Global SHAP), engineers can see which features drive the model's decisions.
- **Example:** An insurance company trains a pricing model excluding "Gender." However, SHAP analysis reveals that "Car Model" and "Profession" have massive importance scores and perfectly correlate with Gender. The XAI audit flags this privacy violation, prompting a retraining with de-biased data.

Red Teaming and Model Inversion

XAI can inadvertently reveal too much about the training data.

- **Application:** Security teams use "Gradient-based Attacks" (essentially XAI techniques) to try to reconstruct private training images from the model.
- **Example:** If an XAI explanation for a facial recognition system reveals pixel-perfect details of a specific individual from the training set, the model has "memorized" private data. This signals a privacy leak, requiring techniques like **Differential Privacy** before release.

4.3.3 Model Development and Debugging

In the MLOps lifecycle, XAI is the primary tool for debugging "Clever Hans" phenomena—where models learn spurious correlations rather than robust features.

Debugging Spurious Correlations

Deep networks are lazy; they choose the easiest path to minimize loss.

- **Application:** Analyzing misclassified samples using LIME or SHAP.
- **Example (The "Husky" Problem):** A famous classifier for "Wolf vs. Dog" had high accuracy as describe above. LIME revealed that for the "Wolf" class, the model was looking exclusively at the **snow** in the background, ignoring the animal. The developers realized their training data had a bias (wolves are usually photographed in snow). Without XAI, this model would have failed catastrophically in a summer environment.

Feature Selection and Pruning

Running large models is expensive. XAI helps optimize inference costs.

- **Application:** Post-training analysis using Permutation Importance or SHAP.
- **Example:** A fraud detection model uses 500 input features. SHAP analysis shows that the bottom 200 features contribute less than 0.1% to the total model output magnitude. Engineers can safely remove these 200 features, reducing the data pipeline latency and storage costs without sacrificing accuracy.

4.3.4 Drift Detection (The "More" Category)

Models degrade over time as the world changes (Data Drift).

- **Application:** Monitoring the distribution of SHAP values over time (Shapley Value Drift).
- **Example:** In e-commerce, "User Age" might be the most important predictor for sales in 2020. In 2023, XAI monitoring shows "Social Media Referrer" has suddenly overtaken "Age" in importance. This shift alerts data scientists that consumer behavior has fundamentally changed, triggering a model retrain sooner than a simple accuracy metric would indicate.

Chapter 5

Uncertainty Quantification—the future of Responsible AI

5.1 The Next Frontier: Uncertainty Quantification

As Deep Learning systems are deployed in safety-critical environments (autonomous driving, medical diagnosis), accuracy is no longer the sole metric of success. A model that predicts "Cancer: No" with 99% confidence when it is actually unsure is far more dangerous than a model that predicts "Uncertain." This brings us to the next milestone in AI research: moving from **Point Estimates** to **Probabilistic Distributions**.

5.1.1 The Illusion of Confidence: Logits vs. True Uncertainty

A common misconception is that the output of a Softmax layer represents probability or confidence. It does not.

The Softmax Fallacy

Standard Deep Learning trains a deterministic function $f_\theta(x)$ to minimize Cross-Entropy. The output probabilities are given by:

$$P(y = k|x) = \frac{e^{z_k}}{\sum_j e^{z_j}} \tag{5.1}$$

The Problem: The Softmax forces the outputs to sum to 1, regardless of the evidence.

- **In-Distribution:** If the model sees a clear image of a "Cat," it might output $[0.9, 0.1]$.
- **Out-of-Distribution (OOD):** If the model sees pure static noise or a dataset it was never trained on, it *should* output $[0.5, 0.5]$ (I don't know). Instead, because

the network is a high-dimensional non-linear mapping, it often produces arbitrarily high logits for one class, outputting $[0.99, 0.01]$ for garbage input.

This phenomenon is known as **Overconfidence**. The model conflates "probability of class k" with "epistemic certainty," which are mathematically distinct concepts.

5.1.2 Types of Uncertainty

To fix this, we must distinguish between two types of uncertainty:

1. **Aleatoric Uncertainty (Data Noise):** Uncertainty inherent in the data itself (e.g., a blurry image, or a coin flip). This cannot be reduced with more data.
2. **Epistemic Uncertainty (Model Ignorance):** Uncertainty due to lack of knowledge. The model has not seen enough training examples in this region of the input space. This *can* be reduced with more data.

Standard Neural Networks model neither well. To capture Epistemic Uncertainty, we need **Bayesian Neural Networks (BNNs)**.

5.1.3 Bayesian Neural Networks and MCMC

In a standard network, weights θ are fixed numbers. In a BNN, weights are **probability distributions**. We seek the **Posterior Distribution** over weights given the data $\mathcal{D}$:

$$P(\theta|\mathcal{D}) = \frac{P(\mathcal{D}|\theta)P(\theta)}{P(\mathcal{D})} \tag{5.2}$$

Prediction is no longer a single forward pass, but an integral (ensemble) over all possible weight configurations:

$$P(y|x, \mathcal{D}) = \int P(y|x, \theta)P(\theta|\mathcal{D})d\theta \tag{5.3}$$

Markov Chain Monte Carlo (MCMC)

The integral above is intractable for deep networks. **MCMC** is the "Gold Standard" method for approximating it. Instead of finding the single "best" set of weights (Optimization), MCMC samples a chain of weights $\theta_1, \theta_2, \ldots, \theta_N$ from the posterior distribution.

$$P(y|x) \approx \frac{1}{N} \sum_{i=1}^{N} P(y|x, \theta_i) \tag{5.4}$$

Pros: It provides the true uncertainty landscape. If the model is unsure, the sampled weights will produce widely varying predictions, resulting in a high-variance (uncertain) output. **Cons:** It is extremely computationally expensive. Storing and sampling thousands of copies of a generic ResNet (25M parameters) is not feasible for real-time applications.

5.1.4 Amortized Inference

To make uncertainty practical, we turn to **Amortized Inference**. Instead of sampling from the true complex posterior $P(\theta|\mathcal{D})$ every time (which is slow), we train a separate "inference network" (or modify our current network) to approximate it using a simpler distribution $q_\phi(\theta)$ (typically a Gaussian).

Variational Inference (VI)

We want to find the parameters ϕ (means and variances of weights) that make our simple distribution $q_\phi(\theta)$ as close as possible to the true posterior $P(\theta|\mathcal{D})$. This is achieved by minimizing the Kullback-Leibler (KL) Divergence, which is equivalent to maximizing the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(\theta)}[\log P(\mathcal{D}|\theta)] - D_{KL}(q_\phi(\theta)||P(\theta)) \tag{5.5}$$

- **Term 1 (Likelihood):** Encourages the weights to explain the data well (Accuracy).
- **Term 2 (Prior Matching):** Encourages the weights to stay close to a prior (Regularization/Uncertainty).

Monte Carlo Dropout

A famous practical example of Amortized Inference is **MC Dropout** (Gal & Ghahramani, 2016). They proved that training a network with Dropout is mathematically equivalent to a specific form of Variational Inference. **Usage:** Instead of turning off Dropout during testing, we keep it **on** and run the forward pass T times.

- Prediction: Mean of the T passes.
- Uncertainty: Variance of the T passes.

If the model is confident (in-distribution), neurons will fire robustly regardless of dropout, and variance will be low. If the model is unsure (OOD), dropout will cause wild fluctuations in the output, signaling high epistemic uncertainty.

5.1.5 Applications of Uncertainty Quantification

The shift from point estimates to probabilistic distributions is not merely an academic exercise. In safety-critical and resource-constrained environments, the ability to quantify "what the model does not know" is an operational requirement.

Autonomous Vehicles: The "Open World" Problem

Self-driving cars are trained on finite datasets, but the real world is infinite.

- **Scenario:** A perception system trained in sunny California encounters a snowy road in Norway (Out-of-Distribution).
- **Without UQ:** The model forces a classification (e.g., predicting "Road" with 99% confidence via Softmax), potentially causing the car to drive off the road.
- **With UQ:** The model detects high **Epistemic Uncertainty** because the input lies far from the training manifold. This triggers a safety fallback protocol: "Disengage Autopilot" or "Slow Down," preventing an accident.

Healthcare: Referrals and Triage

In automated diagnosis, a wrong prediction can be fatal. UQ creates a fail-safe mechanism for Human-AI collaboration.

- **Scenario:** An AI analyzes a chest X-ray for pneumonia.
- **Application:** We set a threshold for uncertainty.
  - *Low Uncertainty:* The AI diagnosis is auto-forwarded to the patient (Efficiency).
  - *High Uncertainty:* The system flags the case for "Expert Review."
- **Benefit:** This focuses scarce doctor attention only on the difficult cases (the "edge cases") rather than routine negatives, optimizing hospital resources while maintaining safety standards.

Active Learning: Efficient Data Labeling

Labeling data (e.g., having a radiologist outline tumors) is expensive and slow. UQ allows models to choose which data they want to learn from.

- **Mechanism:** Train a model on a small seed dataset. Run inference on a massive unlabeled pool.
- **Acquisition Function:** Select the samples where the model exhibits the highest **Epistemic Uncertainty** (i.e., the samples the model is most confused about).
- **Result:** Labeling these specific samples provides maximum information gain. Research shows that Active Learning using UQ can achieve state-of-the-art performance using only 20% of the data required by random sampling.

Reinforcement Learning (Exploration vs. Exploitation)

In robotics, an agent must decide whether to perform a known safe action or try a new, potentially dangerous action to learn more.

- **Application:** Agents use uncertainty as an intrinsic reward signal. "If I am uncertain about the outcome of this action, I should try it to reduce my ignorance."
- **Impact:** This solves the exploration problem, allowing robots to learn complex tasks (like walking or grasping) faster by actively seeking out novel states rather than getting stuck in local optima.

5.1.6 Conclusion: Uncertainty as a Milestone

The transition from deterministic Logits to Amortized Probabilistic Inference marks the maturation of Deep Learning. Just as a human expert says "I'm not sure" when facing a novel situation, the next generation of AI models must quantify their own ignorance. This capability is the prerequisite for **Safe AI**, allowing systems to hand control back to humans when they encounter the unknown, rather than making confident, catastrophic errors.

Bibliography

[1] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[2] G. D. Forney, "The viterbi algorithm," *Proceedings of the IEEE*, vol. 61, no. 3, pp. 268–278, 2005.

[3] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to linear regression analysis*. John Wiley & Sons, 2021.

[4] D. W. Hosmer Jr, S. Lemeshow, and R. X. Sturdivant, *Applied logistic regression*. John Wiley & Sons, 2013.

[5] L. Rokach and O. Maimon, "Decision trees," in *Data mining and knowledge discovery handbook*, pp. 165–192, Springer, 2005.

[6] W. S. Noble, "What is a support vector machine?," *Nature biotechnology*, vol. 24, no. 12, pp. 1565–1567, 2006.

[7] S. R. Eddy, "Hidden markov models," *Current opinion in structural biology*, vol. 6, no. 3, pp. 361–365, 1996.

[8] A. F. Agarap, "Deep learning using rectified linear units (relu)," *arXiv preprint arXiv:1803.08375*, 2018.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[10] L. O. Chua and T. Roska, "The cnn paradigm," *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications*, vol. 40, no. 3, pp. 147–156, 2002.

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.

[12] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*, pp. 234–241, Springer, 2015.

[13] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, "Convnext v2: Co-designing and scaling convnets with masked autoencoders," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 16133–16142, 2023.

[14] P. Bharati and A. Pramanik, "Deep learning techniques—r-cnn to mask r-cnn: a survey," *Computational Intelligence in Pattern Recognition: Proceedings of CIPR 2019*, pp. 657–668, 2019.

[15] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *Proceedings of the IEEE international conference on computer vision*, pp. 2961–2969, 2017.

[16] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2117–2125, 2017.

[17] A. Sherstinsky, "Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, 2020.

[18] Y. Yu, X. Si, C. Hu, and J. Zhang, "A review of recurrent neural networks: Lstm cells and network architectures," *Neural computation*, vol. 31, no. 7, pp. 1235–1270, 2019.

[19] R. Dey and F. M. Salem, "Gate-variants of gated recurrent unit (gru) neural networks," in *2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*, pp. 1597–1600, IEEE, 2017.

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[21] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*, pp. 8748–8763, PmLR, 2021.

[22] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15619–15629, 2023.

[23] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.

[24] M. T. Ribeiro, S. Singh, and C. Guestrin, "" why should i trust you?" explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 1135–1144, 2016.